UNIVERSITY OF PLYMOUTH

PENINSULA COLLEGE

FINTRACK-ANALYSIS: A PERSONAL FINANCE MANAGEMENT SYSTEM

Final Year Project Report

BSC(Hons) Computer Science (Software Engineering)

BSSE2409191

MAL3018

Computing Project

Supervisor: Amir Rasyadan

Acknowledgement

I would like to express my sincere gratitude to my supervisor Sir Amir Rasyadan and my lecturer for their guidance, support and valuable feedback through the development of this project. Their advice has greatly contributed to successful completion of this project.

I would also like to thank my family and friends for their continuous encouragement and support during this project. Their motivation has helped me stay focused and complete my work on time.

Lastly, I would like to thank everyone who has directly or indirectly contributed to this project. Their support is greatly appreciated.

Abstract

Personal financial management has become increasingly important as individual face growing financial responsibility such as managing daily expenses, maintaining savings, and monitoring investments. However, many people lack effective tools that provide a comprehensive overview of their financial activities. Existing financial application often focusses on only one aspect of financial management, such as expense tracking or budgeting, without integrating savings, investment and overall financial health analysis into a single platform. As a result, individual frequently struggle to maintain financial discipline, evaluate their financial progress, and make informed financial decisions.

This project presents **FinTrack-Analysis,** an interactive web based personal finance management system designed to integrate budgeting, savings tracking, investment monitoring and net worth analysis into a single platform. This system is developed using the MERN stack (MongoDB, Express.js, React.js and Node.js) to ensure scalability, security and responsive performance. FinTrack-Analysis enable user to record and categorized financial transaction, track savings growth, monitor investment portfolio and visualize their financial health through interactive dashboards and charts.

The platform also provides analytical insight such as net worth calculation and financial trend visualization, allowing user to better keep understanding their financial behaviour and progress toward financial goals. By consolidating multiple financial management functions within one system, FinTrack-Analysis aims to simplify personal financial tracking and promote improved financial awareness.

The expected outcome of this project is a functional and user-friendly financial tracking application that helps users, particularly students and young professional, develop better financial management habits and make data driven financial decision.

Link

Pabblepad	https://v3.pebblepad.co.uk/spa/#/public/mtw79ZbskqpzsGn8fbG9GnxbwM
Youtube	https://youtu.be/8FgVXOhsmFY
Github	https://github.com/ZULKAMAL12/FinTrack-Analysis

Table of content

Contents

Cover Page 1

Acknowledgement----------2

Abstract 3

Table of Content 4

Introduction 8

1.1 Project Overview 8

1.2 Background of the study 9

1.3 Problem Statement 10

1.4 Project Aim 10

1.5 Project Objective 11

1.6 Scope of the Project 11

1.7 Report Strucute 12

Literature Review. 13

2.1 Introduction 13

2.2 Personal Finance Mangement System 13

2.3 Financial Technology (FinTech) and Personal Finance Applications 14

2.4 Frontend Component Testing 14

2.4.1 Wallet by BudgetBakers 15

2.4.2 YNAB (You Need A Budget) 16

2.4.3 PocketGuard 17

2.4.4 MAE by Maybank 18

2.5 Google Finance as a Financial Information Platform 19

2.6 Financial Data Visualiztion and Dashbaord 20

2.7 Technology used for Financial Tracking System 20

2.8 Summery of Literature Review 21

2.9 Research Gap 22

Methodology 23

3.1 Introduction 23

3.2 Development Methodology 23

3.2.1 Justification for Choosing Agile 24

3.2.2 Agile Application in FinTrack-Analysis 24

3.3 Project Planning Approach 25

3.3.1 Product Backlog 25

3.3.2 Sprint-Oriented Development 26
3.4 System Development Process 26
3.4.1 Requirement Identification 26
3.4.2 System Design 26
3.4.3 Implementation 26
3.5 Tools and Technology Used 27
3.5.1 Fronted Tecnology 27
3.5.2 Backend Technologies 27
3.5.3 Database 27
3.5.4 Data Visualization 27
3.5.5 Development Tools 27
3.6 System Architecture 28
3.6.1 Presentation Layer 28
3.6.2 Application Layer 28
3.6.3 Data Layer 29
3.6.4 Architectutal Justification 29
3.7 Database Design 30
3.7.1 Database Requirements 30
3.7.2 Main Collection 31
3.7.3 Database Design Rationale 33
3.7.4 Relationship Between Collections 33
3.8 Testing Strategy 34
3.8.1 Purpose of Testing 34
3.8.2 Types of Testing Applied 34
3.8.3 Testing Approach 35
3.8.4 Importance of Verification and Validation 35
3.9 Risk and Ethical Consideration in Development 35
3.9.1 Development Risk 36
3.9.2 Ethical and Professional Consideration 36
3.10 Chapter Summary 37
Evaluation 37
4.1 Introduction 37
4.1.1 System Architecture Diagram 37
4.2 System Overview 38
4.3 System Design 38
4.3.1 Use Case Design 38

4.3.2 Use Case Diagram 39
4.3.3 System Module 40
4.3.4 User Interface Design 41
4.3.5 Sequence Diagram for Savings Module 42
4.4 System Implementation 42
4.4.1 Fronted Implementation 42
4.4.2 Backend Implementation 43
4.4.3 Database Implementation 43
4.4.4 ER/ Database Diagram 43
4.4.5 Dashboard Implementation 44
4.4.6 Data Visualization 44
4.5 System Features Implementation 44
4.5.1 Expense Tracking Feature 44
4.5.2 Savings Tracking Feature 44
4.5.3 Investment Tracking Feature 44
4.5.4 Net Worth Calculation 45
4.6 System Integration 45
4.7 Challenges Faced During Implementation 45
4.8 Chapter Summary 45
Testing and Evaluation 46
5.1 Introduction 46
5.2 Testing Objectives 46
5.3 Testing Strategy 46
5.4 Functional Testing 47
5.4.1 Test Cases 47
5.5 Integration Testing 47
5.6 User Interface Testing 48
5.7 Validating Testing 48
5.8 System Evaluation 48
5.8.1 Usability Evaluation 48
5.8.2 Performance Evaluation 48
5.8.3 Functional Effectiveness 49
5.9 Comparison with Exisitng System 49
5.10 Limitation of the System 49
5.11 Chapter Summary 49
Conclusion and Future Work 50

6.1 Introduction 50
6.2 Summary of the Project 50
6.3 Achievement of Objectives 50
6.4 Contribution of the Project 51
6.5 Limitation of the Project 51
6.6 Future Work 51
6.7 Chapter Summary 52
Reference: 54
Appendix A (UI ScreenShot) 55
Appendix B (Code Snippet) 55

1. Introduction

1.1 Project Overview

In today's digital economy, effective personal financial management has become increasingly important for individual to maintain financial stability and achieve long- term financial goals. Many people face difficulties managing their daily expenses, savings, and investment due to the lack of structured financial monitoring tools. While financial awareness has improved over recent years, individuals still struggle to maintain a clear overview of their financial health, including how their income, expense, assets and liability interact to form their overall net worth.

This project introduces FinTrack-Analysis, an interactive personal finance management system designed to help users track and analyse their financial activities in centralized platform. The system integrates multiple financial management function including budgeting, savings tracking, investment monitoring and net worth analysis. By consolidating these financial components into a single web-based application, the system enables users to better understand their financial behaviour and make more informed financial decisions.

The application is developed using the MERN stack (MongoDB, Express.js, React.js and Node.js) to ensure scalability, performance and flexibility for future expansion for future expansion. The system provides users with and intuitive dashboard containing financial summaries, chart and analytical tools that visualize financial trends and patterns. These visualizations allow users to easily identify their spending habits, savings progress and investment growth over time.

In addition to basic financial tracking, the system introduces analytical features designed to support financial planning. For example, the platform calculates a user net worth by aggregating assets and liabilities, providing a clear representation of financial health. The system also enables users to track expense and monitor investment such as stock or cryptocurrency.

The primary objective of FinTrack-Analysis is to promote financial awareness and improve financial discipline by offering comprehensive view of their financial activities. The system aims to support individuals, particularly students and young professional, in developing better money management habits through data driven insights.

1.2 Background of the study

Personal financial management is an essential skill that enables individuals to maintain financial security and plan for future financial goals. Proper financial management involves tracking income, controlling expense, saving money and investing strategically to generate long term wealth. However, many individuals find it difficult to effectively manage this financial aspect due to limited financial literacy or lack of appropriate tools.

Traditional methods of financial management, such as manual budgeting using spreadsheets or handwritten records, are often inefficient and time consuming. These methods can also be prone to human errors, making it difficult for individuals to accurately track their financial activities overtime. As a result, many individuals lose visibility of their financial status and may struggle to maintain financial discipline.

Modern financial technology (FinTech) application has attempted to address this issue by providing digital solutions for expense tracking and budgeting. However, many existing solutions focus primary on one aspect of financial management, such as expense monitoring or budgeting without providing complete overview of user financial situation.

Furthermore, some financial management platforms require user to possess advanced financial knowledge to fully utilize their feature. This can discourage beginner users who simply want a straightforward and easy to understand system for managing their finance.

In Malaysia, financial literacy among young adults remains growing concern. Many student and early career professional struggle with manging their finances effectively, often due to lack of awareness and limited access to comprehensive financial management tools. This issue highlights the need for a simple yet powerful financial tracking system that can assist users in managing their financial activities more effectively.

FinTrack-Analysis addresses these challenges by providing and integrated financial tracking platform that combines multiple financial management features within a single system. By offering budgeting tools, savings monitoring, investment tracking and financial visualization, the system aims to simplify financial management and encourage better financial habits.

1.3 Problem Statement

Manging personal finances has become increasingly complex due to the growing number of financial activities individual must handle. Many individual use multiple account, investment platforms and digital payment system, making it difficult to track all financial transactions in one place.

Several key challenges exist in personal financial management:

First, many individuals struggle to **track their financial activities across multiple accounts and expense categories**. Without a centralized platform, it becomes difficult to obtain a complete overview of income, expense and financial commitments.

Second, maintaining **consistent saving discipline** is another challenge. Individuals often fail to monitor their savings progress or compare their financial behaviour against predefined financial goals.

Third, monitoring **investment performance** can be complicated, especially for beginner investor who may not have access to simplified investment tracking tools. Investments such as stock and cryptocurrencies often require separate platforms, making it difficult for users to evaluate their overall investment portfolio.

Another significant issue is the lack of **visibility into overall financial health**. Many individuals do not have a clear understanding of their net worth, which includes assets such as savings and investments as well as liabilities such as debts.

Finally, individual often struggle to **evaluate the affordability of major purchases**. Many purchasing decisions are made based only on monthly payment rather than considering the long-term financial impact.

Due to these challenges, individual may experience poor financial planning, overspending, and delayed financial goals. Existing financial tools often fail to address all these problems within a single integrated platform.

Therefore, these is a need for comprehensive financial management system that integrates budgeting, savings tracking, investment monitoring and net worth analysis in a single platform. FinTrack-Analysis is designed to address this issue by providing users with accessible, data driven financial tracking system that improves financial awareness and decision making.

1.4 Project Aim

The aim of this project is to design and develop an interactive web based personal finance management system that integrates budgeting, saving tracker, investment monitoring and net worth analysis into a single platform.

The system aims to provide users with centralized environment where they can monitor their financial activities, visualize financial trends and make informed financial decisions.

1.5 Project Objective

To achieve the project aim, the following objective are defined:

1. To design and develop a web based personal finance tracking system that integrate budgeting, savings, investment and net worth analysis.
2. To implement financial data visualization through dashboards and charts to provide uses with clear insight into their financial activities.
3. To integrate external APIs for tracking investment data such as stock price and cryptocurrency values.
4. To implement secure authentication and user data protection mechanism to ensure privacy and security.
5. To design a responsive and user-friendly interface that is accessible across multiple devices.

1.6 Scope of the Project

The scope of this project defines the features and limitation of the FinTrack-Analysis system.

In-scope Features

The system includes several core modules designed to provide comprehensive financial tracking:

- Income and expense tracking
- Budget planning and monitoring
- Savings tracking including capital and dividend growth
- Investment portfolio tracking for stock and cryptocurrencies
- Debt and loan management
- Net worth calculation based on assets and liabilities
- Financial dashboard with data visualization charts
- Secure user authentication and account management

Out of Scope Features

Certain functionality is excluded from the scope of the current project due to technical and time limitation.

- Direct integration with banking system
- Automated tax calculation or tax filing
- Fully automated financial planning or advisory services
- Advance AI financial forecasting beyond affordability analysis

Future versions of the system may incorporate additional features such as automated financial recommendation and advance predictive financial analytics.

1.7 Report Structure

This report is organized into several chapters to present the development process and evaluation of the FinTrack-Analysis system.

- **Chapter 1: Introduction** – Provides background information, problem statement, project aims, objectives, and scope.
- **Chapter 2: Literature Review** – Reviews existing personal finance management systems and relevant technologies.
- **Chapter 3: Methodology** – Describes the development methodology, tools, technologies, and system architecture.
- **Chapter 4: System Design and Implementation** – Explains system design, database structure, and implementation details.
- **Chapter 5: Testing and Evaluation** – Presents system testing, results, and performance evaluation.
- **Chapter 6: Discussion and Analysis** – Analyses the effectiveness of the system and compares results with project objectives.
- **Chapter 7: Conclusion and Future Work** – Summarizes the project outcomes and suggests improvements for future development.

2. Literature Review

2.1 Introduction

Personal finance has become increasingly important in the modern digital economy. Individuals must manage multiple financial responsibility such as tracking daily expenses, maintaining savings, monitoring investment and planning long term financial goals. However, manging these financial activities can be challenging without appropriate tools that provide financial insights.

With the rapid growth of financial technology (FinTech), various digital platforms have been developed to help users track and manage their finances more effectively. These platforms provide feature such as expense tacking, budgeting tools, investment monitoring and financial analytics dashboard. Despite the availability of these tools, many existing systems focus on only on aspect of financial management, which limits their usefulness for users seeking a comprehensive financial overview.

This chapter reviews the existing research and technology related to personal finance management system. It also analyses several widely used financial applications and platforms such as Mint, YNAB (You Need a Budget), Budget Banker, and Google Finance. These systems are examined to identify their features, strength, and limitations. The insights obtained from this review help justify the design and development of FinTrack-Analysis system, which aims to integrate multiple financial management functions into a single web-based platform.

2.2 Personal Finance Management Systems

Personal finance management system are digital tools designed to help individuals monitor, analyse, and manage their financial activities. These systems typically provide features such as expense tracking, budgeting tools, savings management, and financial reporting.

According to Investopedia (2023), personal finance management involves planning and managing financial activities such as budgeting, saving, investing and managing debt. Effective financial management enables individuals to achieve financial stability and long-term financial goals.

Many people struggle with financial management due to a lack of financial literacy or limited access to effective financial tools. Without proper financial tracking, individuals may overspend, fail to maintain consistent savings, or make poor financial decision. Digital financial management system aims to address these issues by providing automated tracking, financial visualization and decision support tools.

Modern financial tracking application often provides dashboards that display financial summaries, spending patterns, and savings progress. These dashboards allow users to visualize their financial data using charts and graphs, making it easier to understand financial trends and identify areas for improvement.

However, despite the availability of such tools, many existing systems do not provide a complete view of an individual's financial situation. Some applications focus primarily on expense tracking while others focus mainly on budgeting or investment monitoring. This limitation highlights the need for more integrated financial management platform.

2.3 Financial Technology (FinTech) and Personal Finance Applications

Financial Technology, commonly referred to as FinTech, refers to the use of digital technologies to improve financial services. FinTech solutions have transformed the financial industry by introducing innovative applications for banking, payment, lending, investment management and financial analytics.

FinTech applications enable individuals to access financial services through digital platforms such as mobile applications and web-based systems. These applications often include features such as transaction monitoring, automated budgeting, investment tracking, and financial forecasting.

According to the EOCD (2023), financial technology has played a significant role in improving financial literacy by providing user with tools that help the understand their financial behaviour. By analysing financial data presenting it through visual dashboards, FinTech platforms allow user to gain insight into their spending patterns and financial progress.

The development of FinTech application has also increased the accessibility of financial management tools. User no longer need advanced financial knowledge to monitor financial activities. Instead, modern financial platforms provide intuitive user interface and automated analytics that simplify financial decisions-making.

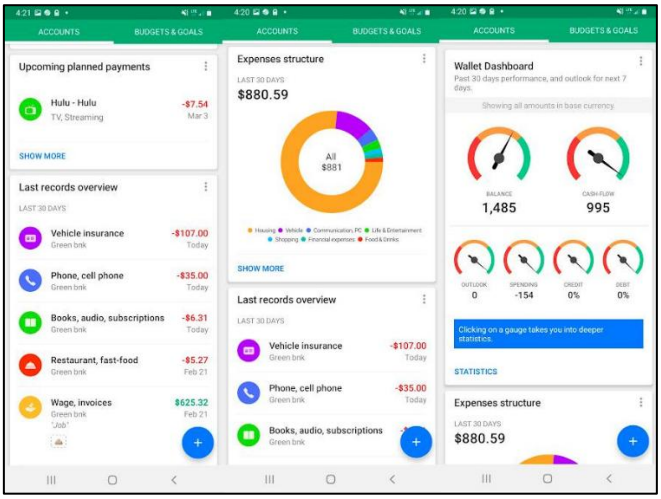
However, many FinTech platforms still face limitation in providing comprehensive financial analysis. Some platforms are restricted to specific regions or making financial institutions, while other require user to manually input data, which can reduce convenience and usability.

These limitation highlights the importance of developing a flexible and user-friendly financial management system that can integrate multiple financial tracking feature within a single platform.

2.4 Existing Personal Finance Applications

Several personal finance management systems currently exist that help users track their financial activities. Some mostly widely known applications include Wallet by BudgetBakers, YNAB, and PockerGuard.

2.4.1 Wallet by BudgetBakers



Wallet by BudgetBakers is widely used personal finance management applications designed to help user's monito their financial activities, manage budgets, and track expense across multiple accounts. The application allows users to record financial transactions, categorize expenses, and generate financial reports that help users understand their spending behaviour.

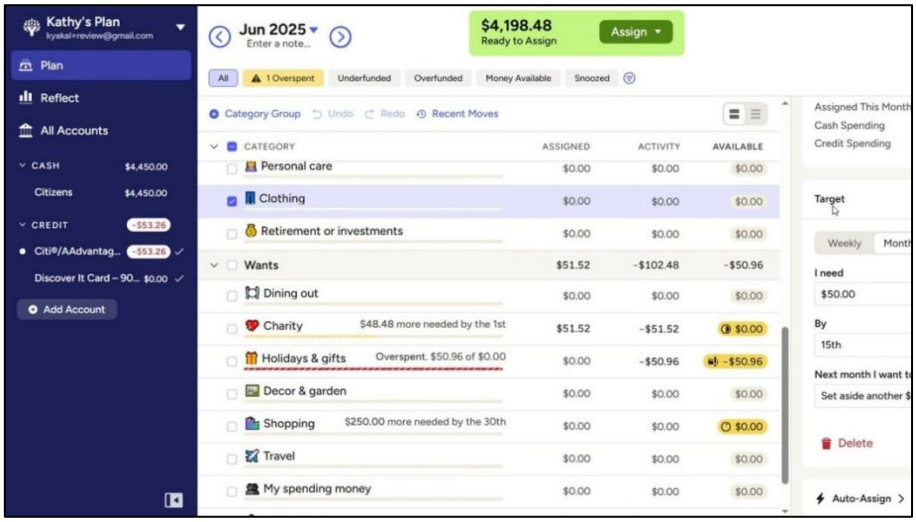
One of the key features of Wallet is its ability to synchronize financial data from different accounts and display financial information through visual chart and reports. The system providing budgeting tools that allow user to set financial goals and track their progress over time. In addition, the platform includes financial analysis features that help users identify spending patterns and valuate their financial habits.

Another advantage of Wallet is its user-friendly interface, which allows users to easily navigate through different financial features. The system supports multiple currencies and accounts, making it suitable for individuals who manage finance across different financial platforms.

However, despite these useful features, Wallet has certain limitations. The application primarily focuses on expense tracking and budgeting, while advanced financial analysis features such as integrated investment portfolio monitoring and detailed net worth analysis are limited. In addition, some advanced features require a premium subscription, which may discourage some user from fully utilize the platform.

These limitations highlight the need for a more integrated financial management system that combine budgeting, savings tracing, investment monitoring, and net worth analysis within a single platform. The FinTrack-Analysis system aims to address this limitation by providing a comprehensive financial tracking solution with integrated analytics and financial visualization tools.

2.4.2 YNAB (You Need A Budget)

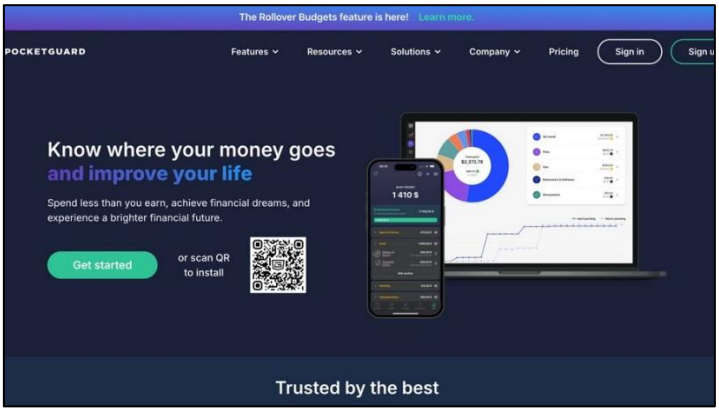


YNAB is another well-known personal finance application that focuses on primarily on budgeting. The system encourages users to allocate every unit of income to specific spending categories, which promotes disciplined budgeting behaviour.

The YNAB budgeting methodology users plan their financial activities in advance rather than simply tacking past spending. This proactive approach encourages users to maintain financial discipline and avoid overspending.

Despite its effective as budgeting tool, YNAB has several limitations. The platform requires a subscription fee, which may discourage some users from adopting the system. Additionally, YNAB focus primarily on budgeting and does not provide extensive investment monitoring or net worth analysis features.

2.4.3 PocketGuard

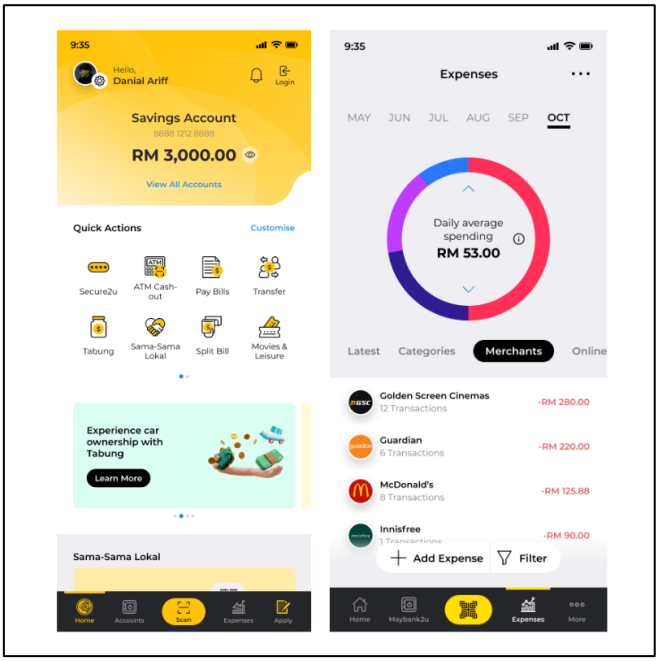


PocketGaurd is a personal finance management application that help users track their spending and identify available funds after accounting for bills. Savings goals, and essential expenses.

One of PockerGuard's key features is its ability to calculate how much disposable income users have after covering their financial commitments. These features help users avoid overspending and maintain better control over their finance.

However, like other financial tracking applications, PocketGuard mainly focus on budgeting and expense monitoring. It does not provide comprehensive financial analysis tools such as integrated tracking or detailed net worth calculations.

2.4.4 MAE by Maybank



MAE by Maybank2u is a digital financial application developed by Maybank. The application is widely used in Malaysia and providers users a range of financial services including digital payments, expense tracking and budgeting tools.

One of the key features of MAE is its expense tracking and spending insights, which allow users to monitor their financial transactions and analyse their spending patterns. The system automatically categorizes transactions made through the Maybank account and present the data through visual chart and summaries. This feature hep users understand where their money is being spent and encourages better financial planning.

MAE also provides additional financial tools such as budget planning, tabung (savings goals) features, and financial activity summaries. These features enable users to set financial goals and monitor their progress toward achieving those goals. The integration with both banking services allows transactions to be updated automatically, reducing the need for manual financial record keeping.

Despite these useful features, MAE has certain limitation as a personal financial management system. The platform is primarily designed for Maybank account holders, which limit its accessibility for users who utilize multiple financial institutions. In addition, while MAE provides basic expense racing and budgeting tools, it does not offer comprehensive financial analysis features such as detailed investment tracking, portfolio performance monitoring, or complete net worth analysis.

These limitations highlight the opportunities for systems such as FinTrack-Analysis to provide a more comprehensive financial management platforms that integrate budgeting, savings tracking, investment monitoring and financial analysis into a single unified system.

2.5 Google Finance as a Financial information platform

Google Finance is a financial information platform developed by Google that provides real-time financial market data, stock price tracking, and financial news. The platform allows users to monitor stock market performance and track investment portfolios through a simple and user-friendly interface.

Google Finance provides users with access to various financial indicators such as stock prices, historical performance charts, market trends, and financial news updates. These features make it useful resource for investors who want to monitor the performance for their investments.

One of the strengths of Google Finance is its ability to present financial data through interactive charts and visualizations. User can view price trends, compare multiple stocks, and analyse financial data over different time periods. This visualization approach makes it easier for users to understand market movement and evaluate investment opportunities.

However, Google Finance is primarily designed as financial data platform rather than comprehensive personal finance management system. While it allows user to track investment portfolios, it does no provide features such as expense tracking, budgeting tools, or savings management.

Despite the limitation, Google Finance serve as an important reference platform for financial data visualization and investment monitoring. The visualization and portfolio tracking features of Google Finance provide useful inspiration for the investment component of the FinTrack-Analysis system.

In the FinTrack-Analysis project, Google Finance is used as a reference for designing financial dashboard and investment tracking features. By studying how financial data is presented in Google Finance, the project aims to create similar visualization that help users understand their financial activities and investment performance.

2.6 Financial Data Visualization and Dashboard

Financial dashboards play a critical role in modern financial management systems. A financial dashboard is a visual interface that present financial data using charts, graphs, and summary statistics.

Dashboard allow user to quickly understand their financial status without analysing large volumes of numerical data. For example, charts can display monthly spending patterns, savings growth or investment performance over time.

Data visualization tools such as Chart.js D3.js and Google Charts are commonly used to build financial dashboards in web-based applications. These tools allow developers to create interactive chart that update dynamically as financial data changes.

Effective financial dashboards should be designed to present information clearly and intuitively. Users should be able to quickly identify key financial metrics such as income, expense, savings progress and investment performance.

The FinTrack-Analysis system incorporates financial dashboard to help users visualize their financial activities. These dashboards include chart that displays spending patterns, savings growth and investment performance, enabling user to gain better insight into their financial behaviour.

2.7 Technology Used for Financial Tracking System

The development of modern financial tracking systems often relies on full-stack web technology that support scalability, performance and flexibility.

One widely used technology stack for web applications is the MERN stack, which consists of MongoDB, Express.js, React.js and Node.js.

MongoDB is NoSQL database that store data in flexible JSON like documents, making it suitable for applications that require dynamic data structures. Express.js provide a lightweight backend framework for building web APIs and handling server-side logic.

React.js is a front-end library used to build interactive user interface. It allows developers to create dynamic components that update automatically when data changes. Node.js serve as the runtime environment allow JavaScript to run on the server side.

The MERN stack is particularly suitable for building real-time web-based application that require interactive dashboard and dynamic data updates. This makes it ideal technology choice for the development of the FinTrack-Analysis system.

2.8 Summery of Literature Review

This chapter reviewed several personal finance management system and financial technology platforms related to financial tracking and investment monitoring. Applications such as Wallet by BudgetBakers, YNAB (You Need A Budget), and PocketGuard provide useful tools for users to manage their personal finances. These systems mainly focus on helping users track expenses, create budgets, and monitor their spending behaviour. By providing visual summaries and financial insights, these applications assist users in their financial awareness and managing their daily financial activities.

In addition, the review examined MAE by Maybank, a Malaysian digital banking application that include goal tracking. MAE demonstrate how banking application can integrate financial monitoring tools to help users better understanding their financial behaviour. However, the system is mainly designed for users within Maybank ecosystem and focuses primarily on banking transactions rather than providing a full financial analysis platform.

Furthermore, Google Finance was analysed as a financial information platform that provides investment monitoring and financial market data. Google Finance allow users to track stock prices, analyse market trends and monitor investment portfolio through interactive charts and financial indicators. While the platform provides useful tools for investment tracking and financial data visualization, it does not offer comprehensive personal finance management features such as budgeting, expense tracking, or savings monitoring.

Overall, the analysis of existing system reveal that many financial applications focus on specific aspects of financial management rather than providing a complete overview of an individual financial situation. Most system specialize in either budgeting, expense tracking, or investment monitoring, but rarely integrate all these functions within a single platform. As a result, users often need to rely on multiple applications to manage different aspects of their finances.

These limitations highlight the need for a more integrated financial management system. The FinTrack-Analysis project aims to address this gap by developing a web-based platform that combine budgeting, expense tracking, savings monitoring, investment tracking, and net worth analysis within a single system. By integrating these features and presenting financial data through interactive dashboards and visualizations, FinTrack-Analysis seeks to provide user with a clearer understanding of their overall financial health and support better financial decisions making.

2.9 Research Gap

The literature review shows that many existing financial management applications provide useful support for users, but most of them are designed to solve only part of the overall personal finance problem. Application such as Wallet by BudgetBakers, YNAB, PockerGuard, and MAE by Maybank focus mainly on budgeting, expense tracking, or bank related spending insight. On the other hand, Google Finance is more suitable for monitoring market data and investment performance rather than managing compete personal finances. AS a result, users often need to depend on multiple platforms to gain a broader understanding of their financial situation.

One major gap identified is the lack of integration across financial functions. Existing systems often separate budgeting, savings, investment motoring, and net worth tracking into different tools. This fragmented approach makes it difficult for users to understand how one part of their financial activity effects another. For example, user may be able to monitor monthly expenses in one system and view stock performance in another, but still not have a clear overall picture of their financial health.

Another gap is limited support for holistic financial analysis. Many systems provide transaction summaries and categories expense reports, but fewer offer combined analysis of assets, liabilities, savings growth and investment performance. Without this integrated view, users may understand isolated financial activities but still struggle to evaluate their total financial position or progress toward long term goals.

A further is that many existing platforms are region specific or ecosystem specific. For example, MAE by Maybank is useful for Malaysian user with Maybank ecosystem, but its not designed as a universal financial analysis system for users managing multiple accounts, savings platforms, or investment channels. Similarly, some international applications are built mainly around financial system and services available in other countries, which reduces their relevance for local users.

There is also a gap in term of customized financial insight for everyday users, especially students and young adults. Many people do not need highly advanced financial software, but they do need a system that is simple, clear and able to combine the most important financial functions in one place. Existing applications may either be too limited in scope or too dependent on external banking integration and subscriptions.

Therefore, the key research and development gap addressed by this project is the need for a unified, user-friendly, web-based financial analysis platform that combines expense tracking, budgeting, savings monitoring, investment tracking, and net worth analysis. The proposed FinTrack-Analysis system is designed to address this gap by integrating this component into a single application supported by dashboard, chart and analytical summaries. This gives users a more complete understanding of their financial position and supports better financial planning and decision making.

3. Methodology

3.1 Introduction

This chapter explains the methodology used to plan, design, develop and evaluate the FinTrack-Analysis system. Since the project involves the development of a web-based software application, the methodology must support iterative development, continuous improvement, and regular testing though the project lifecycle. In addition to software construction, the methodology must also support project planning, backlog management, feature prioritization, and review of progress.

For this project, an Agile software development approach, specifically using iterative and incremental development principles, was selected. This approach was chosen because it allows the system to developed in manageable stages while enabling improvements to be made continuously as new requirements, design ideas, and implementation challenges emerges. This suitable for FinTrack-Analysis because the project contains multiple interrelated modules, such as budgeting, expense tracking, savings monitoring, investment tracking, and net worth analysis, which can be developed and refined progressively.

This chapter also describes the project planning approach, development tools and technologies, system architecture, design process and testing strategy used in the project. The aim is to demonstrate that the system was not developed in an ad hoc manner, but through a structured and justified methodology aligned with software engineering practices.

3.2 Development Methodology

The development of FinTrack-Analysis followed an Agile methodology. Agile is software development approach that emphasizes flexibility, iterative progress, collaboration, and continuous feedback. Instead of completing the whole system in one linear sequence, Agile breaks development into a smaller cycles or iterations, allowing features to be designed, implemented, tested and refined over time.

This approach was suitable for FinTrack-Analysis for several reasons. First the system contains several functional modules that can be developed independently and then integrated into a larger platform. For example, the expense tracking module, savings module, investment module, and analysis dashboard can be developed in separate iterations. Second, Agile supports changes in requirements, which is useful because the project evolved during development while still maintaining the original baseline proposed in the project proposal. Third, Agile encourages regular testing and review, which helps reduce implementation errors and improve system quality.

In this project, the Agile methodology was applied through a simplified sprint-based workflow. The workflow. The work was organized into stages such as planning, design, implementation, testing, and review, but these stages were repeated across different system modules rather than treated as one time sequence. Each development cycle focused on a specific feature or module and produced a functional improvement to the system,

The usage of Agile also align well with the project brief, which emphasizes planning, backlog management, sprint-based progress and iterative project control. The brief expects evidence of project vision, risk assessment, Gantt chart planning, and evolving implementation supported by version control and project management.

3.2.1 Justification for Choosing Agile

Agile was chosen over more traditional models such as the waterfall model because waterfall is less flexible when project requirements evolve during development. In Waterfall, the project usually moves from requirement to design, implementation, testing and development in a strict linear sequence. While this can work well for projects with fixed and fully defined requirements, it is less suitable for a project like FinTrack-Analysis, where interface design, dashboard components, features priorities, and data relationships were refined during the implementation process.

Agile provides several advantages for this project:

- It provides incremental delivery, allowing the system to be built feature by feature.
- It allows changes and refinements to be made when better design decisions are discovered.
- It encourages continuous testing, reducing the risk if major issue appearing only at the final stage.
- It improves project manageability, since complex features can be broken into smaller development tasks.
- It matches the practical nature of a final year project, where learning, experimentation, and improvement happen alongside implementation.

For these reasons, Agile was considered the most appropriate methodology for developing FinTrack-Analysis.

3.2.2 Agile Application in FinTrack-Analysis

In practice, Agile was applied by dividing the system into functional modules and developing them iteratively. The Project began with initial planning and identification of core requirements based on the project proposal. After that, the development moved through a series of iterations, where each iteration focused on implementing and refining one or more core modules.

Example of iterative development in this project include:

- Designing and refining the dashboard to display financial summaries and visual insights.
- Building the expense tracking module with category support and monthly analysis.
- Developing the savings module with capital, dividend and withdrawal handling.
- Implementing the investment tracking module with monthly records and portfolio growth tracking.
- Creating the net worth and analysis features to combine assets, liabilities and performance indicators.
- Improve usability through interface refinement, responsive layout design and better chart visualization.

Each stage involved planning the feature, implementing it, testing it, identifying issue, and improving it before moving to the next stage. This iterative workflow allowed the project to remain organized while still being flexible enough to incorporate improvements through the development process.

3.3 **Project Planning** Approach

Effective project planning was necessary to ensure that FinTrack-Analysis could be completed within the allocated timeframe while meeting both technical and academic requirements. The project planning process included defining the project vision, identifying core features, prioritizing tasks, estimating stages and considering risks that could affect progress.

The project was initially guided by the project proposal, which defined the system concept, objectives, baseline features, and intended technology stack. From this starting point, the work was translated into development tasks and milestones. Core modules such as budgeting, expense management, savings tracking, investment tracking, and analytics were identified as key deliverables.

Planning was supported through the following project management elements:

- A **project vision**, which defined the purpose and intended value of the system.
- A **feature backlog**, listing the main system modules and enhancements.
- A **Gantt chart style schedule**, used to estimate when major phases of the project will be completed.
- **Version control**, used to track progress and maintain a history of implementation.
- **Risk assessment**, used to identify possible technical and project related problems early.

This planning approach supported the agile methodology by giving the project an overall direction while still allowing flexibility in how and when individuals features were implemented.

3.3.1 Product Backlog

The product backlog is prioritized list of system features, tasks, and improvements to be developed during the project. In FinTrack-Analysis, the backlog evolved during the project as new requirements and refinements were identified. However, the core backlog items remained consistent with the original project direction.

Key backlog items included:

1. User authentication and account management
2. Dashboard with financial summaries
3. Expense and income tracking
4. Budget planning and comparison
5. Savings tracking with capital, dividend and withdrawal records
6. Investment tracking with monthly records and performance monitoring
7. Debt and liability tracking
8. Net worth calculation
9. Data visualization using chars
10. Analysis and reporting features
11. Responsive user interface improvement
12. Testing, validation and bug fixing

These backlog items were not always completed in a strict sequence. Instead, they were implemented and refined according to development needs, technical dependencies, and usability priorities.

3.3.2 Sprint-Oriented Development

Although the project was carried out individually rather than in a full commercial team environment, the development still followed a sprint-oriented mindset. Work was grouped into manageable periods focusing on specific modules or sets of related features.

A simplified example of the sprint structure is as follows:

- Sprint 1: Project setup, environment preparation, initial structure, authentication baseline.
- Sprint 2: Expense tracking and related calculations.
- Sprint 3: Savings tracking and related calculations.
- Sprint 4: Investment module and performance records.
- Sprint 5: Dashboard integration and chart visualization.
- Sprint 6: Analysis, net worth calculation, refinement and testing.

This approach allowed progress to be reviewed incrementally and reduced the risk of leaving all testing and integration to the final stage.

3.4 System Development Process

The development process for FinTrack-Analysis followed several repeated steps within each iteration. These steps are summarized below.

3.4.1 Requirement Identification

The first step involved identifying the functional and non-functional requirements of the system. Functional requirements included the ability to record expenses, track savings, monitor investments, calculate net worth, and visualize data through dashboards. Non-functional requirements included usability, responsiveness, maintainability, and security.

Requirement were derived from:

- The original project proposal
- Personal use-case expectations for finance tracking
- Comparative analysis of existing systems
- Refinement identified during development

3.4.2 System Design

After requirements identification, the system structure and interface design were planned. This included defining the major modules, navigation flow, page layouts, database entities, and data relationships. Wireframe-level thinking and progressive UI refinement were used to shape the system interface.

The design process considered both functionality and usability. Since the project aimed to support everyday users, it was important that the interface remain clear, organized, and visually understandable.

3.4.3 Implementation

The implementation stage involved converting design decisions into working code. Fronted components were developed for user interaction, while backend services were created to manage data processing and persistence. MongoDB collections were corrected before new features were added.

This iterative testing approach improved reliability and helped ensure that previously developed features continued to work after new changes were introduced.

3.5 Tools and Technologies Used

The FinTrack-Analysis system was developed using a modern full-stack web development environment. The selected tools and technologies were chosen based on suitability, familiarity, scalability and compatibility with the project objectives.

3.5.1 Fronted Technologies

The fronted of the system was developed using React.js, with supporting technologies such as HTML, CSS, JavaScript, and modern UI styling approaches. React was selected because it supports component-based development, dynamic rendering, and efficient user interface updates. This is particularly useful for dashboards and financial data displays where values and charts change based on user interaction and stored records.

3.5.2 Backend Technologies

The backend was developed using Node.js and Express.js. Node.js enables JavaScript to be used on the server side, while Express.js provides a lightweight framework for building APIs and handling application routing. These technologies were appropriate because they integrate well with React and Support efficient full-stack JavaScript development.

3.5.3 Database

The system uses MongoDB as its primary database. MongoDB was selected because it store data in flexible document-based structures, which are suitable for financial records that may contain slightly different attributes across modules such as expenses, savings, investment, and debts.

3.5.4 Data Visualization

Charts and dashboards visualization were implemented using web-based charting tools such as Chart.js. This supported the creation of bar charts, line charts, pie charts and summary visualizations that help users understand financial patterns more clearly.

3.5.5 Development Tools

Development was supported by tools such as:

- Visual Studio Code for code editing
- GitHub for version control
- GitHub Actions or deployment pipeline where relevant
- Postman for API testing
- Docker / Docker compose where used for environment consistency
- Browser developer tools for frontend debugging

These technologies provided a complete development environment suitable for building, testing, and maintaining the system.

3.6 System Architecture

The architecture of FinTrack-Analysis was designed using a layered full-stack web application structure. This architecture separates the system into distinct components so that each part has a clear responsibility. A layered structure improves maintainability, support future enhancement, and reduces the risk of tightly coupled code. It also aligns with good software engineering practices, which important for demonstrating implementation quality in the project.

The system architecture consists of three main layers:

1. Presentation Layer
2. Application Layer
3. Data Layer

3.6.1 Presentation Layer

The presentation layer is the frontend interface that users interact with directly. This layer was developed using React.js and is responsible for displaying pages, forms, dashboards, charts, tables and navigation elements. Its main role is to collect user input and present processed financial information in a clear and user-friendly format.

Example of frontend components in FinTrack-Analysis included:

- Login and authentication pages
- Dashboard summary card
- Expense and Budget pages
- Savings and investment pages
- Net worth analysis
- Data tables and charts

The user of React.js supports reusable components, making it easier to manage the interface and maintain consistency across the system. This especially useful for modules that share similar structures, such as financial forms, record tables, and summary sections.

3.6.2 Application Layer

The application layer contains the server-side logic of the system. It was developed using Node.js and Express.js. This layer receives requests from the frontend, processes business logic, validate data, performs calculations, and communicates with the database.

The application layer is responsible for handling operations such as:

- User authentication and session-related logic
- Creating, updating, retrieving, and deleting financial records
- Calculating totals and monthly values
- Computing net worth and summary metrics
- Preparing structured data for charts and dashboard
- Supporting filtering and report logic

By centralizing this logic in the backend, the system reduces duplication and ensures that financial calculations remain consistent across different pages and modules.

3.6.3 Data Layer

The data layer is responsible for storing and managing persistent system data. For this project, MongoDB was used as the database. MongoDB stores records in document form, which fits well with the flexible structure required by the system. Since the application includes multiple finance-related modules with slightly different data requirements, a document-based database provides greater flexibility than a rigid relational schema.

The data layer store information such as:

- User accounts
- Expense and Budgeting records
- Savings records
- Investment records
- Debt or liability records
- Transaction records
- Dashboard summary data derived from stored values

The separation of these three layers improves the structure of the system and makes future development easier. For example, the frontend can be improved without changing the core database structure, while new backend calculation can be added without redesigning the entire user interface.

3.6.4 Architectural Justification

The chosen architecture was appropriate because FinTrack-Analysis is not a simple static application. It requires dynamic user interaction, storage of large amounts of personal finance data, calculation of summaries, and visual presentation through charts and dashboards. A layered MERN-based architecture allows these requirements to be handled effectively while supporting modular development and iterative enhancement.

This structure also helps demonstrate that the system is more than a basic CRUD application. It includes interaction between frontend, backend, and database layers, along with computed financial analysis and user-cantered dashboard functionality, which support a stronger academic and technical justification of the system implementation.

3.7 Database Design

The database of FinTrack-Analysis was developed to support multiple finance-related modules within a unified system. Because the project combines expense tracking, budgeting, savings, investment, debts and accounts, the database needed to support different types of financial records while maintaining logical relationships between them.

MongoDB was used as the primary database technology. Instead of using fixed relational tables, MongoDB stores data in collections of documents, This approach was suitable because it allows financial records to be stored with flexible attributes and nested structures where needed.

3.7.1 Database Requirements

The database was designed to support the following functional needs:

- Storing registered user data securely
- Saving income and expense record by category and date
- Recording savings value such as capital, dividend and withdrawal
- Recording investment values and monthly performance
- Tacking debt and repayment records
- Providing data dashboard summaries and chart visualizations.

These requirements meant that the database had to handle both transactional data and summary-related financial data.

3.7.2 Main Collections

The system database includes several major collections. Although the exact implementation may evolve during the development, the main collection is as follows:

a. User Collection

The user's collection stores account-related information for each user. This includes login Credentials, profile, profile details, and any required security-related information.

Typical fields include:

- User ID
- Name
- Email
- Password hash
- Created date

This collection forms the base identity layer of the system because all financial records are associated with a specific user.

b. Expense Collection

The expense collection stores both expense and income records, depending on how the system handles financial entries. Each record contains information about the amount of category, date, account, used and transaction type.

Typical fields include:

- User ID
- Type (income or expense)
- Category
- Amount
- Date
- Account
- Description

This collection supports monthly summaries category analysis, and dashboard reporting.

c. Savings Collection

The savings collection stores savings-related records. Since the project supports cumulative monthly savings tracking, this collection may store total capital, dividend and withdrawals by account and month.

Typical field include:

- User ID
- Account name
- Month
- Year
- Total capital
- Total dividend
- Total withdrawal
- Created or update date

This structure supports tracking savings growth overtime and enables historical comparison.

d. Investment Collection

The investment collection stores investment-related records, including capital contributed, dividends or returns, and withdrawals. Like savings module, this may use a monthly model where one record represents the state of an investment platform for a given month.

Typical fields include:

- User ID
- Platform name
- Month
- Year
- Capital
- Return or dividend
- Withdrawal
- Notes

This collection supports calculation of investment growth and dashboard trends.

e. Debts Collection

The debts collection stores liabilities, including short-term and long-term debts. This allows the system to compute net worth more accurately by subtracting liabilities from total assets.

Typical fields include:

- User ID
- Debt name
- Type
- Amount
- Outstanding balance
- Payment history
- Due date

This collection supports financial analysis and debt tracking.

3.7.3 Database Design Rationale

The database was structured around finance modules rather than placing all data in a single mixed collection. This design improves clarity, maintainability, and query organization. It also makes it easier to apply module-specific logic and perform analysis on different types of records.

For example:

- Expense records need category-based and month-based filtering.
- Savings records need cumulative calculations over time.
- Investment records need return tracking and historical trend comparison.
- Debt records need outstanding balance monitoring.

Separating collections allows each module to evolve without causing excessive complexity in others.

3.7.4 Relationship Between Collections

Although MongoDB is non-relational, logical relationships still exist between collections through shared user identifiers and account references. For example:

- Each user can have multiple expenses, savings entries, investments, debts, and accounts.
- Expense records may reference an account used for the transaction.
- Savings and investments may reference a specific platform or account.
- Dashboard analytics combine data from several collections to generate summaries.

This means the system uses document-based storage while still following clear logical relationships like those found in traditional database design.

3.8 Testing Strategy

Testing is a critical part of software development because it helps ensure that the system functions correctly, produces accurate results, and provides a reliable user experience. In FinTrack-Analysis, testing was performed throughout development rather than being reserved for the final stage. This aligns with the iterative nature of Agile development and supports the continuous refinement of system quality. The project brief also emphasizes appropriate testing, verification, and validation as part of project evaluation.

3.8.1 Purpose of Testing

The main objectives of testing in this project were:

- To verify that each module functions as intended
- To validate that calculations and summaries are accurate
- To identify bugs, logic errors, and interface issues
- To ensure the system handles common and edge-case inputs appropriately
- To improve reliability and user confidence in the system

Because FinTrack-Analysis includes financial calculations, testing was especially important for ensuring the correctness of totals, balances, growth values, withdrawals, and net worth results.

3.8.2 Types of Testing Applied

Several forms of testing were used during the development of the project.

a. Functional Testing

Functional testing was used to check whether individual features performed according to the intended requirements. This included verifying that users could add, edit, delete, and view records correctly across modules such as expenses, savings, investments, and debts.

Examples include:

- Checking that an expense record is saved with the correct amount and category
- Verifying that a savings record updates the displayed totals
- Confirming that investment records appear correctly in charts and history sections
- Ensuring that transfers update account balances appropriately

b. Integration Testing

Integration testing was performed to ensure that different modules worked together correctly. Since the system combines several finance components, it was necessary to confirm that data from one module did not conflict with another and that dashboard summaries reflected changes across the system.

Examples include:

- Confirming that dashboard values update after expenses or savings are added
- Checking that net worth calculations reflect both asset and debt changes

- Verifying that account-based transactions are reflected in linked modules

c. User Interface Testing

User interface testing focused on the visual and interactive aspects of the system. This included checking whether pages were responsive, forms were usable, buttons worked correctly, and charts displayed readable information.

Since the project targets general users rather than expert analysts, usability was an important factor. The interface had to be understandable, consistent, and accessible across common screen sizes.

d. Validation Testing

Validation testing ensured that user inputs were checked before being accepted into the system. This helped prevent invalid or incomplete financial records from being stored.

Examples include validating:

- Required fields
- Numeric amount inputs
- Category selection
- Month and year consistency
- Prevention of empty or clearly invalid entries

3.8.3 Testing Approach

Testing was carried out progressively during implementation. After each major feature or module was developed, it was tested individually before integration with the broader system. Where issues were identified, corrections were made before continuing with further development.

This progressive testing approach was beneficial because it reduced the accumulation of unresolved bugs and made debugging easier. It also allowed the project to maintain a stable base while new features were added.

3.8.4 Importance of Verification and Validation

Verification and validation were both important in this project.

- **Verification** focused on whether the software was built correctly according to the design and logic.
- **Validation** focused on whether the software met the intended user needs and project objectives.

For example, a dashboard may be verified by checking that calculations run correctly, but it is validated by determining whether the user can understand the displayed financial summary and use it to make decisions.

This distinction is important because FinTrack-Analysis is not only a technical system but also a decision-support tool for personal finance management.

3.9 Risk and Ethical Considerations in Development

Risk management and ethical consideration are important parts of software project development. The project brief specifically requires due consideration of legal, ethical, social, and professional issues, along with risk planning.

3.9.1 Development Risks

Several risks were relevant during the development of FinTrack-Analysis:

a. Scope Expansion

Because the project includes many financial modules, there was a risk of adding too many features and exceeding the available development time. This was managed by prioritizing core features first and treating advanced features as enhancements.

b. Data Logic Errors

Financial systems require accurate calculations. Errors in logic, especially around totals, net worth, or monthly growth, could reduce trust in the system. This risk was addressed through repeated testing and validation of calculations.

c. Integration Complexity

As more modules were added, there was a risk that features would become difficult to integrate cleanly. This was reduced by using a modular architecture and separating frontend, backend, and data responsibilities.

d. Time Constraints

As a final year project, the work had to be balanced alongside academic deadlines and other responsibilities. The use of planning tools, backlog prioritization, and iterative progress reduced this risk.

3.9.2 Ethical and Professional Considerations

Although FinTrack-Analysis is primarily a development project, it still involves important ethical and professional responsibilities.

a. Data Privacy

The system stores personal financial information, which is sensitive in nature. Therefore, it is important that user data is handled securely and not exposed unnecessarily. Authentication and controlled access are important in reducing unauthorized access to financial information.

b. Responsible Financial Presentation

Because the system presents financial summaries and insights, the displayed information should be accurate and not misleading. Poor calculations or unclear financial dashboards could negatively affect user decisions. This creates a professional responsibility to ensure the system communicates financial information clearly.

c. Academic Integrity and AI Use

The assignment brief allows assisted use of generative AI for planning, structuring, language refinement, technical guidance, debugging support, and testing support, but requires transparency and declaration of such use. Accordingly, AI support used in the project should be documented appropriately in the final report and declaration appendix.

3.10 Chapter Summary

This chapter described the methodology used to develop FinTrack-Analysis. An Agile, iterative development approach was selected because it allowed the system to be built progressively while supporting refinement, feedback, and continuous testing. The chapter explained the project planning approach, sprint-oriented development style, system architecture, database structure, testing strategy, and the main development risks and ethical considerations.

Overall, the chosen methodology provided a structured yet flexible framework for developing a multi-module financial analysis system. This methodology supported not only the technical implementation of the software, but also the project management and evaluation requirements expected in the final year computing project

4. System Design and Implementation

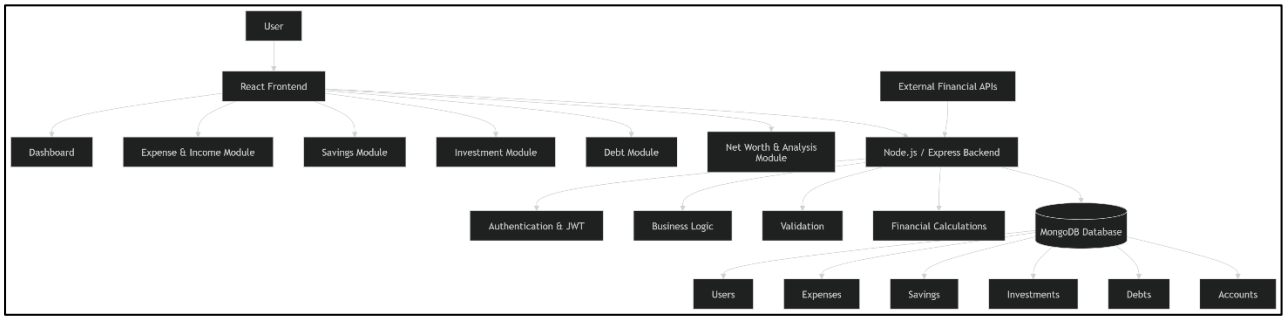
4.1 Introduction

This chapter presents the design and implementation of the **FinTrack-Analysis system**. It explains how the system was structured, designed, and developed based on the methodology described in Chapter 3. The chapter focuses on translating system requirements into a functional application, including system design decisions, user interface development, module implementation, and integration of different components.

The system was developed as a **web-based financial management platform** using the **MERN stack (MongoDB, Express.js, React.js, Node.js)**. The implementation emphasizes modular design, interactive dashboards, and real-time financial data updates. Each module of the system was designed and implemented incrementally following the Agile approach.

This chapter also describes key system components such as the user interface, dashboard, expense tracking, savings management, investment tracking, and financial analysis features. The aim is to demonstrate that the system is fully functional, well-structured, and capable of supporting comprehensive financial tracking and analysis.

4.1.1 System Architecture Diagram



4.2 System Overview

FinTrack-Analysis is designed as a **comprehensive personal finance management system** that integrates multiple financial components into a single platform. Unlike traditional financial applications that focus on only one aspect, this system combines:

- Expense and income tracking
- Budget planning
- Savings management
- Investment tracking
- Debt tracking
- Net worth calculation
- Financial analysis and visualization

The system provides a **centralized dashboard** where users can view their financial status, including total assets, liabilities, savings growth, and investment performance.

The main objective of the system is to provide users with a **clear and complete overview of their financial health**, allowing them to make better financial decisions.

4.3 System Design

The design of FinTrack-Analysis focuses on **usability, modularity, and clarity**. The system is structured into several interconnected modules, each responsible for a specific financial function.

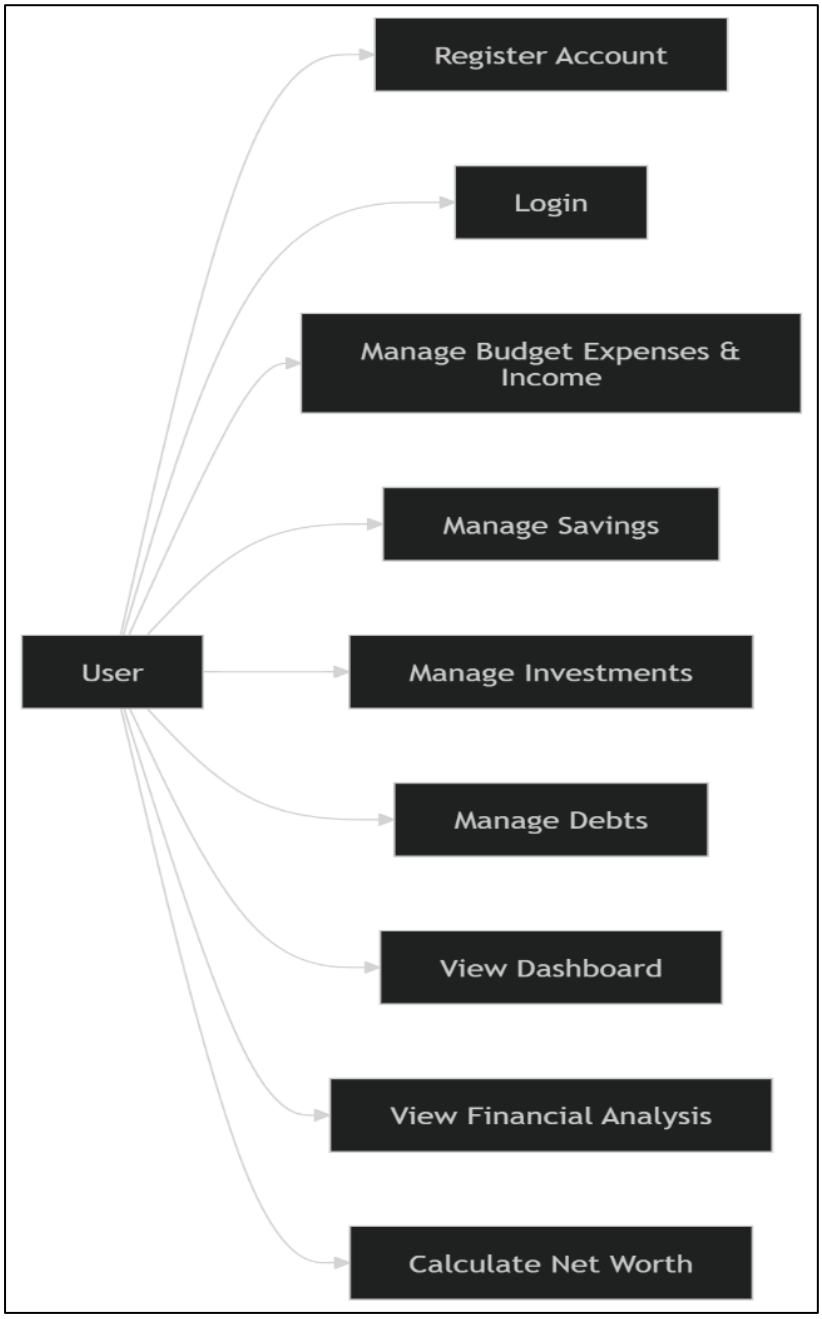
4.3.1 Use Case Design

The system supports a primary user (individual user), and the main use cases include:

- Register and login
- Add, edit, and delete financial records
- View dashboard summary
- Track expenses and income
- Manage savings and investments
- Monitor debts
- View financial analysis and reports

These use cases ensure that users can perform all necessary financial management tasks within the system.

4.3.2 Use Case Diagram



4.3.3 System Modules

The system is divided into the following modules:

a. Authentication Module

Handles user registration and login. Ensures that only authorized users can access financial data.

b. Dashboard Module

Displays financial summaries such as:

- Total balance
- Savings
- Investments
- Debts
- Net worth

Includes charts for quick insights.

c. Expense and Income Module

Allows users to:

- Add transactions
- Categorize expenses
- Track monthly spending
- Compare income vs expenses

d. Savings Module

Tracks:

- Capital (amount saved)
- Dividends (returns)
- Withdrawals

Supports monthly tracking and growth visualization.

e. Investment Module

Tracks:

- Investment capital
- Returns/dividends
- Withdrawals
- Platform-based tracking

f. Debt Module

Tracks liabilities including:

- Outstanding balance
- Payment history
- Short-term and long-term debts

g. Analysis Module

Provides:

- Expense distribution charts
- Financial trends
- Category breakdown
- Monthly comparisons

4.3.4 User Interface Design

The user interface was designed to be:

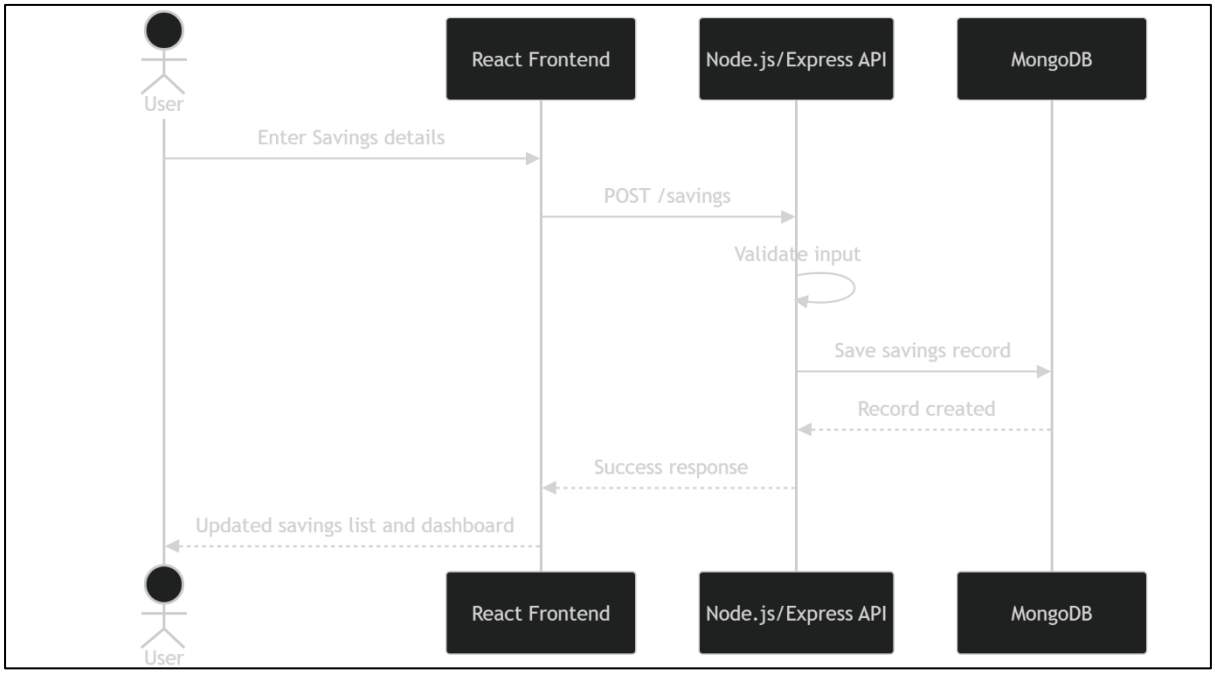
- **Simple and clean**
- **Easy to navigate**
- **Visually informative**

The system uses:

- Dashboard cards for summaries
- Tables for records
- Charts for visualization
- Forms for input

Special attention was given to ensuring that financial data is presented clearly so that users can easily understand their financial status.

4.3.5 Sequence Diagram for Savings Module



4.4 System Implementation

This section explains how the system was built.

4.4.1 Frontend Implementation

The frontend was developed using **React.js**. The application uses a component-based structure, where each feature is divided into reusable components.

Examples include:

- Dashboard component
- Expense form component
- Savings table component
- Chart components

State management is used to update the interface dynamically whenever financial data changes.

4.4.2 Backend Implementation

The backend was implemented using **Node.js and Express.js**. It handles:

- API requests
- Data validation
- Business logic
- Financial calculations

RESTful APIs were created to support operations such as:

- Adding records
- Retrieving financial data
- Updating entries
- Deleting records

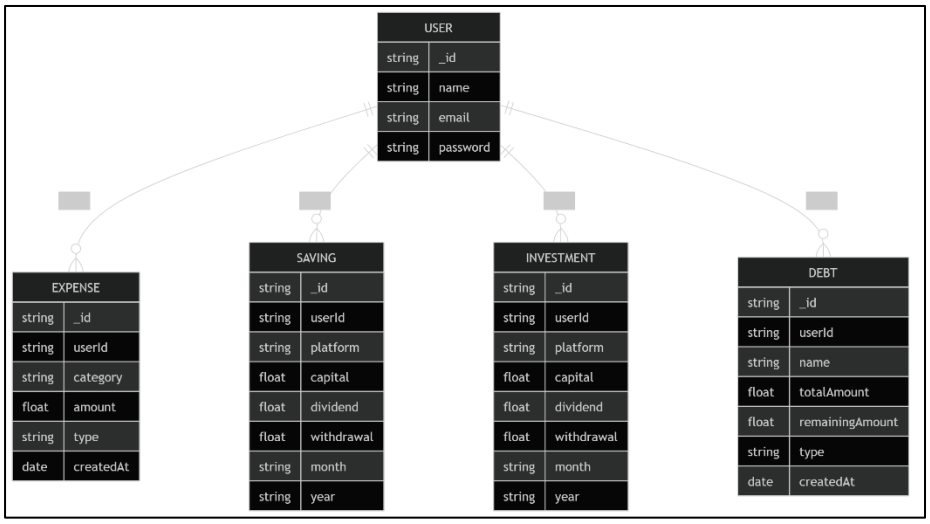
4.4.3 Database Implementation

The system uses **MongoDB** to store financial data. Collections were created for:

- Users
- Expenses
- Savings
- Investments
- Debts

Each collection is structured to support efficient querying and analysis.

4.4.4 ER/ Database Diagram



4.4.5 Dashboard Implementation

The dashboard is one of the core components of the system. It aggregates data from multiple modules and displays it in a unified interface.

Key features include:

- Total financial summary
- Net worth calculation
- Monthly overview
- Charts for trends

The dashboard updates dynamically based on user data.

4.4.6 Data Visualization

Charts were implemented using **Chart.js** to provide:

- Pie charts (expense distribution)
- Bar charts (monthly comparison)
- Line charts (growth trends)

These visualizations help users understand financial patterns more effectively.

4.5 System Features Implementation

This section highlights key implemented features.

4.5.1 Expense Tracking Feature

- Add/edit/delete expenses
- Category-based tracking
- Monthly filtering

4.5.2 Savings Tracking Feature

- Monthly cumulative tracking
- Capital vs dividend separation
- Withdrawal tracking
- ROI calculation

4.5.3 Investment Tracking Feature

- Platform-based tracking
- Monthly updates
- Growth tracking

- Withdrawal handling

4.5.4 Net Worth Calculation

Net worth is calculated as:

Net Worth = Total Assets – Total Liabilities

Where:

- Assets = Savings + Investments
- Liabilities = Debts

4.6 System Integration

The system integrates multiple modules into a single platform. Data flows between modules to produce meaningful insights.

Examples:

- Expenses affect dashboard summaries
- Savings affect total assets
- Debts affect net worth
- Investments affect financial growth

This integration ensures that users see a **complete financial picture** rather than isolated data.

4.7 Challenges Faced During Implementation

Some challenges encountered include:

- Managing multiple interconnected modules
- Ensuring accurate financial calculations
- Designing a clean and understandable dashboard
- Handling data updates across different components
- Maintaining performance with increasing data

These challenges were addressed through iterative development and continuous testing.

4.8 Chapter Summary

This chapter described the design and implementation of the FinTrack-Analysis system. It explained how the system was structured into modules, how each component was developed, and how different technologies were integrated to build a functional application.

The system successfully combines multiple financial management features into a single platform, providing users with a comprehensive view of their financial activities. The implementation demonstrates the practical application of software engineering principles, including modular design, full-stack development, and interactive data visualization.

5. Testing and Evaluation

5.1 Introduction

This chapter presents the testing and evaluation of the **FinTrack-Analysis system**. The purpose of this chapter is to demonstrate that the system functions correctly, meets its requirements, and provides a reliable and usable platform for personal financial management.

Testing was conducted throughout the development process following the Agile methodology described in Chapter 3. Instead of performing testing only at the final stage, each module was tested incrementally as it was implemented. This approach helped identify and resolve issues early, improving the overall quality and stability of the system.

In addition to functional testing, the system was also evaluated in terms of usability, performance, and effectiveness in supporting financial decision-making. The evaluation focuses on whether the system meets its objectives and provides meaningful value to users.

5.2 Testing Objectives

The main objective of testing in this project was:

- To verify that all the system features function correctly
- To ensure financial calculations are accurate
- To validate user inputs and prevent errors
- To test system integration across modules
- To evaluate usability and user experience
- To identify and fix bugs before final deployment

Because the system involves financial data, accuracy and reliability are critical. Incorrect calculations or data inconsistencies could lead to misleading financial insight.

5.3 Testing Strategy

The testing strategy used in this project followed a multi-level testing approach, including:

- Functional Testing
- Integration Testing
- User Interface Testing
- Validation Testing

Testing was performed manually using different scenarios and inputs to simulate real user behaviour.

5.4 Functional Testing

Functional testing ensures that each feature of the system operates according to its intended functionality

5.4.1 Test Cases

Below is sample test cases used during system testing:

Test ID	Features	Test Description	Expected Result	Status
TC01	Login	User enters valid login credentials	User successfully logs in	Pass
TC02	Login	User enters invalid credentials	Error message Displayed	Pass
TC03	Expense	Add new expense records	Record saved and displayed	Pass
TC04	Expense	Delete expense record	Record deleted	Pass
TC05	Savings	Add savings data	Data reflected in summary and chart	Pass
TC06	Investment	Add investment record	Record saved correctly	Pass
TC07	Dashboard	View dashboard summary	Correct financial data displayed	Pass
TC08	Net Worth	Calculate net worth	Accurate value displayed	Pass
TC09	Delete	Delete financial record	Record removed from system	Pass
TC10	Transfer	Transfer between accounts	Balances updated correctly	Pass

These test cases confirm that the system core features function as expected.

5.5 Integration Testing

Integration testing ensures that different modules work together correctly.

Key Integration Scenarios:

- Adding expense updates dashboard totals
- Savings updates effect total assets
- Investment reflects in financial trends
- Debts reduce net worth calculation
- Account transfer update balances across modules

Results showed that the system successfully integrates all modules, providing a unified financial overview.

5.6 User Interface Testing

User interface testing focuses on usability interaction.

Key Aspect Tested:

- Navigation between pages
- Form input usability
- Button functionality
- Chart readability
- Responsive design

The system interface was tested on different screen sizes to ensure it remains usable and visually clear.

Result:

The interface is simple, and user-friendly, making it suitable for students and general users.

5.7 Validating Testing

Validating testing ensures that incorrect or invalid data is not accepted by the system.

Example:

- Empty input fields are rejected
- Non-numeric values for amount are not allowed
- Invalid dates are prevented
- Required fields must be filled

This prevents data inconsistencies and improves system reliability.

5.8 System Evaluation

Beyond testing, the system was evaluated based on its effectiveness and performance.

5.8.1 Usability Evaluation

The system is designed to be:

- Easy to use
- Visually clear
- Organized into logical sections

Users can quickly understand their financial status through the dashboard and charts.

5.8.2 Performance Evaluation

The system performs efficiently under normal usage conditions:

- Data loads quickly
- Charts render smoothly
- CRUD operations respond instantly

Since the system is design for personal use, performance requirement are moderate and well satisfied.

5.8.3 Functional Effectiveness

The system successfully achieves its objectives:

- Tracks expenses and income
- Monitors savings and investments
- Calculate net worth
- Provide financial insights

This confirms that FinTrack-Analysis meets its intended purpose.

5.9 Comparison with Existing Systems

Based on the literature review in chapter 2, the system was compared with existing applications.

Feature	Wallet	YNAB	MAE	Google Finance	FinTrack
Expense Tracking	✓	✓	✓	✗	✓
Budgeting	✓	✓	✓	✗	✓
Savings Tracking	Limited	✓	✓	✗	✓
Investment Tracking	Limited	✗	✗	✓	✓
Net worth	Limited	✗	✗	✗	✓
All in one system	✗	✗	✗	✗	✓

This comparison shows that FinTrack-Analysis provides a more integrated solution.

5.10 Limitation of the system

Despite its functionality, the system has some limitations:

- No real-time bank API integration
- Manual data entry required
- Limited external financial data integration
- Designed primarily for individual's users

These limitations can be improved in future development.

5.11 Chapter Summary

This chapter presented the testing and evaluation of the FinTrack-Analysis system. The system was tested using multiple approaches including functional, integration, interface, and validation testing. The results show that the system functions correctly and provides accurate financial calculations.

The evaluation demonstrates that the system is user-friendly, performs efficiently and successfully achieves its objectives. Compared to existing financial applications, FinTrack-Analysis offers a more comprehensive and integrated solution.

6. Conclusion and Future Work

6.1 Introduction

This chapter concludes the FinTrack-Analysis project by summarizing the overall work carried out, evaluating the achievement of project objectives, and discussing possible future improvements. The aim is to reflect on the development process and highlight the contributions of the system in supporting personal financial management.

6.2 Summary of the Project

The FinTrack-Analysis system was developed as web-based personal finance management platform that integrates multiple financial tracking features into a single application. The system was designed to address the limitations of existing financial tools, which often focus on only one aspect of financial management.

The project began with identifying key problems in personal finance management, including fragmented financial tools, lack of comprehensive financial analysis, and limited data visualization. Based on these issues, the system was designed to provide a unified platform that combines expense tracking, budgeting, savings management, investment monitoring, and debt tracking.

The system was implemented using the MERN Stack, allowing for a scalable and interactive web application. The development process followed an Agie methodology, enabling incremental progress, continuous testing, and iterative improvements throughout the project life cycle.

Key features of the system include a centralized dashboard, financial record management, data visualization through charts and net worth calculations. These features provide users with a clear overview of their financial status and help support better financial decision making.

6.3 Achievement of Objectives

The objective defined In Chapter 1 were successful achieved.

- The system supports expense and income tracking, allowing users to manage their financial transactions effectively.
- A savings module was implemented, including tracking of capital, dividends and withdrawal.
- The investment tracking features enables users to monitor their financial growth over time.
- A debt tracking module allows users to record liabilities and monitor outstanding balances.
- The system successfully calculates net worth by combining assets and liabilities.
- An interactive dashboard was developed to visualize financial data through charts and summaries.
- The system was designed to be user-friendly, ensuring accessibility for general users.
- Testing and evaluation confirmed that the system functions correctly and meet its intended purpose.

Overall, the project achieved its aim of developing a comprehensive financial analysis system.

6.4 Contributions of the Project

The FinTrack-Analysis system contributes to both practical and academic areas.

From a practical perspective, the system provides users with a centralized financial management platform that simplifies financial tracking and improves financial awareness. By integrating multiple financial components, the system helps users to understand their overall financial health rather than viewing isolated financial data.

From an academic perspective, the project demonstrates the application of software engineering principles, including system design, Agile development, full-stack implementation, database design and testing strategies. It also highlights the use of modern technologies to solve real-world problems.

In addition, the system introduces an integrated approach that combines budgeting, savings, investment, and net worth analysis within a single platform, addressing gaps identified in existing financial applications.

6.5 Limitation of the project

Despite its successful implementation, the system has several limitations.

- The system does not support real-time integration with banking system, requiring users to manually input financial data.
- External financial data sources, such as livestock market updates, are limited to the system.
- The system is designed for a single user use and does not support multi-user environments or collaborative financial management.
- Advance financial forecasting and AI-based recommendations are not included in the current version.

These limitations provide opportunities for future improvements.

6.6 Future Work

Several enhancements can be considered to improve the system in future development.

Firstly, the system can be enhanced by integrating bank APIs to allow automatic synchronization of financial transactions. This would reduce the manual data entry and improve user convenience.

Secondly, the addition of real-time data integration, such as stock market updates, would improve the investment tracking module and provide more accurate financial insights.

Thirdly, the system can be extended with AI-based financial analysis, such as spending predictions, budgeting recommendations, and financial health scoring. This would provide users with more intelligent decision-support features.

Fourthly, the system can be expanded to support mobile applications, allowing users to access their financial data more conveniently and more mobile friendly.

Finally, additional features such as multi-user support, goal-based financial planning, different currency tracking and automated reporting can further enhance the functionality and usability of the system.

6.7 Chapter Summary

This chapter summarized the FinTrack-Analysis project and evaluated its outcomes. The system successfully achieved its objective by providing a comprehensive and user-friendly financial management platform. While some limitations exist, the project provides a strong foundation for future development and enhancement.

Overall, FinTrack-Analysis demonstrates the effective application of software engineering techniques to develop a practical solution for personal financial management.

AI Declaration

This report was developed with the assistance of artificial intelligence (AI) tools, including ChatGPT,Gemini to support idea generation, structuring, and language refinement. The use of AI was limited to enhancing clarity and organization of the content.

All work presented in this report, including the system design, implementation, analysis, and conclusions, is the original work of the author. The author has reviewed, edited, and verified all content to ensure accuracy, originality, and compliance with academic integrity requirements.

7.0 Reference

Investopedia. (2023) *Personal Finance Definition and Importance.* Available at: https://www.investopedia.com/terms/p/personalfinance.asp (Accessed: 2 September 2025).

Statista. (2024) *Financial Literacy in Malaysia – Statistics and Insights.* Available at: https://www.statista.com/topics/8038/financial-literacy-in-malaysia/ (Accessed: 5 September 2025).

MongoDB. (2024) *Introduction to the MERN Stack: Building Modern Web Applications.* Available at: https://www.mongodb.com/mern-stack (Accessed: 14 September 2025).

OECD. (2023) *Improving Financial Literacy and Consumer Protection in Asia.* Available at: https://www.oecd.org/finance/financial-education/ (Accessed: 23 September 2025).

PwC Global. (2024) *How Artificial Intelligence is Transforming Financial Services.* Available at: https://www.pwc.com/gx/en/industries/financial-services/publications/ai-in-finance.html (Accessed: 1 October 2025).

Google Finance (2026) *Google Finance: Market Summary and Financial Data*. Available at: https://www.google.com/finance (Accessed: 10 Feb 2026).

Versa Asia (2025) *Versa Investment Platform Overview*. Available at: https://versa.com.my (Accessed: 15 Feb 2026).

Bank Negara Malaysia (2025) *Financial Consumer Alert and Financial Education Resources*. Available at: https://www.bnm.gov.my (Accessed: 12 March 2026).

Amanah Saham Nasional Berhad (ASNB) (2025) *ASB Investment Information*. Available at: https://www.asnb.com.my (Accessed: 15 March 2026).

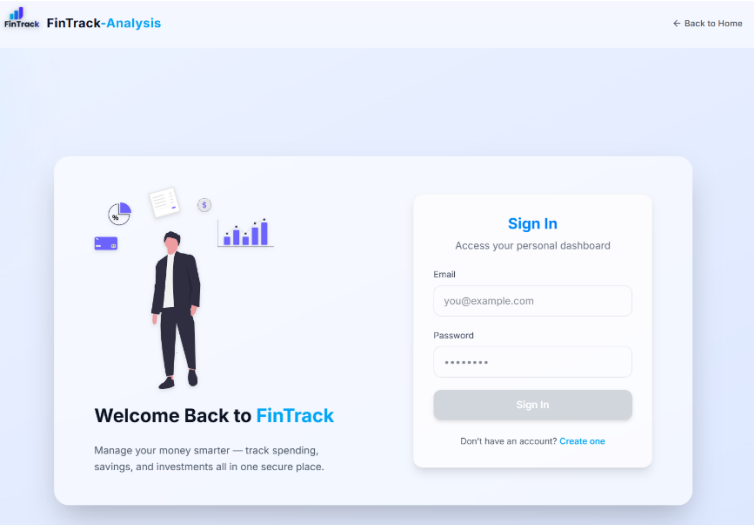
W3Schools (2025) *Web Development Technologies Guide*. Available at: https://www.w3schools.com (Accessed: 20 March 2026).

Yahoo Finance (2025) *Financial News and Market Data*. Available at: https://finance.yahoo.com (Accessed: 10 March 2026).

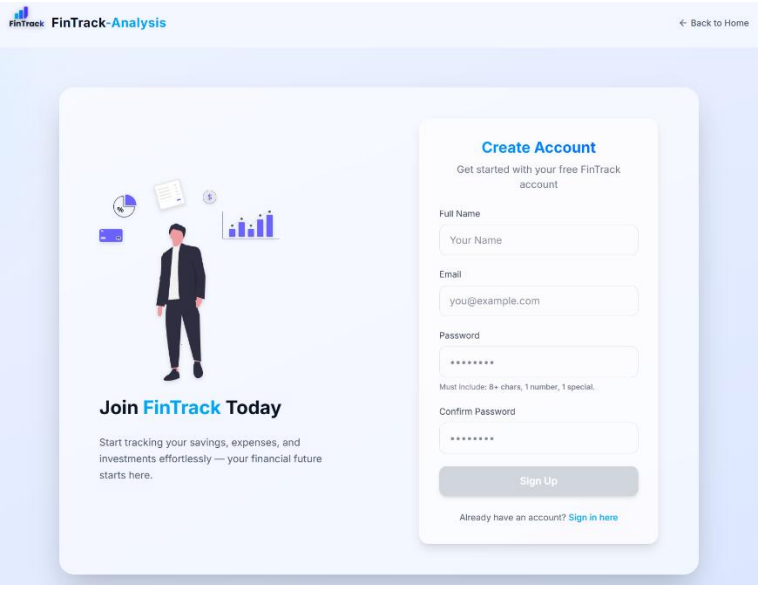
Appendix A – UI Screen Shot

Screen shot of Landing pages

Screen shot of Sign In Pages



Screen shot of Registration Pages



Screen shot of Dashboard pages

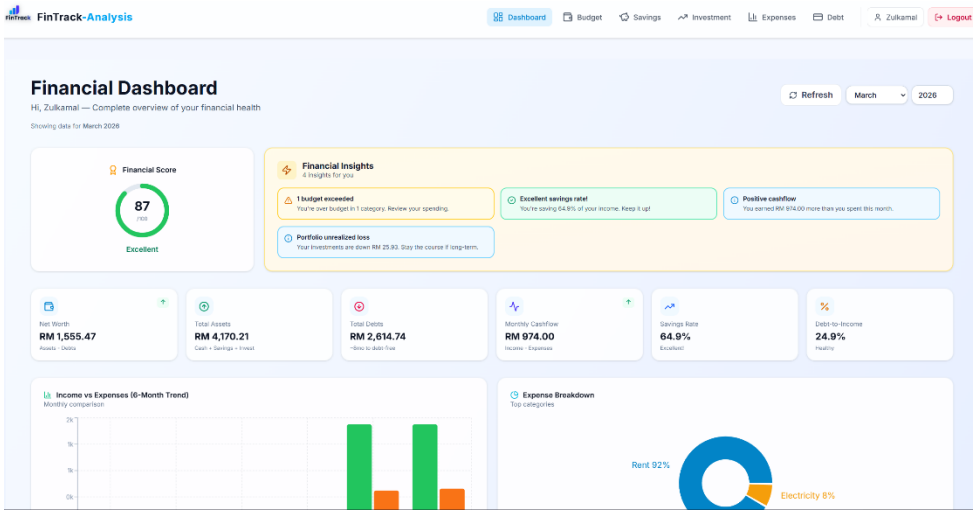
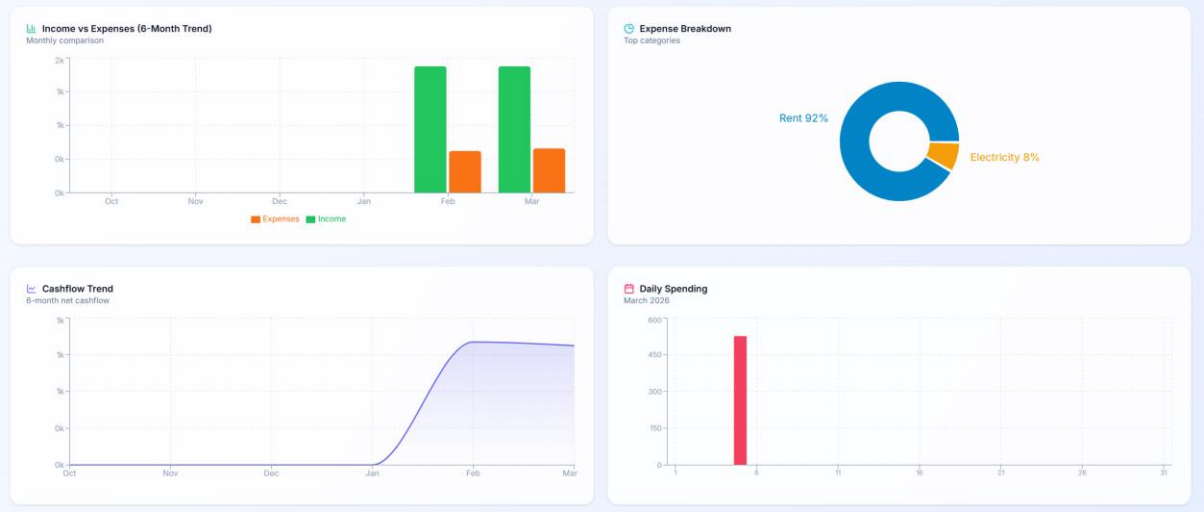
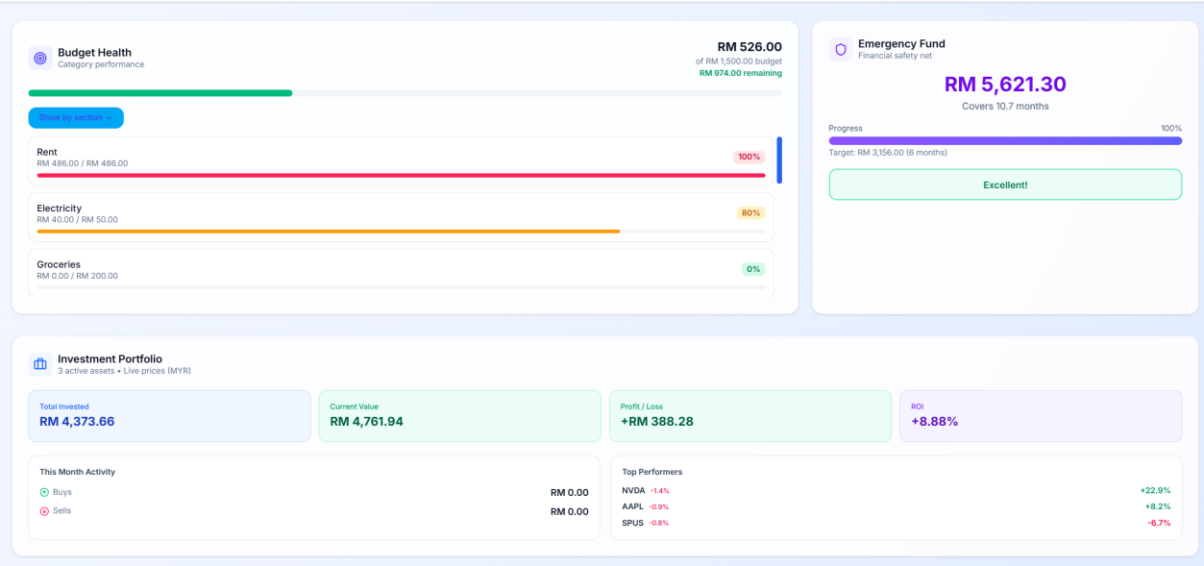


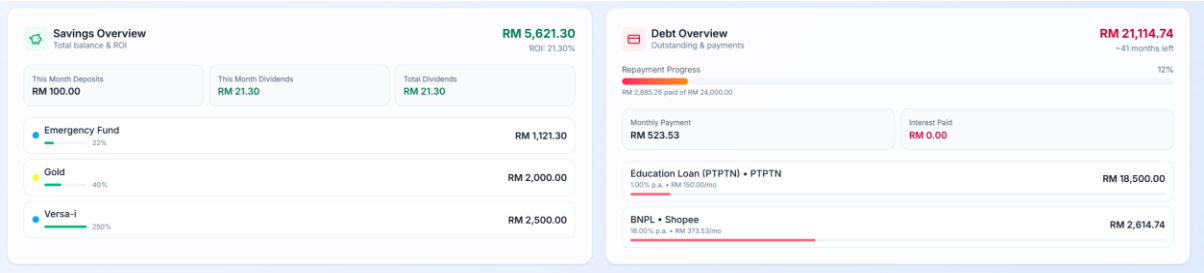
Chart for the Dashboard pages



Budget health, emergency Fund and Investment portfolio for the Dashboard



Savings overview and Debt overview of dashboard pages



Screen shot of Saving pages

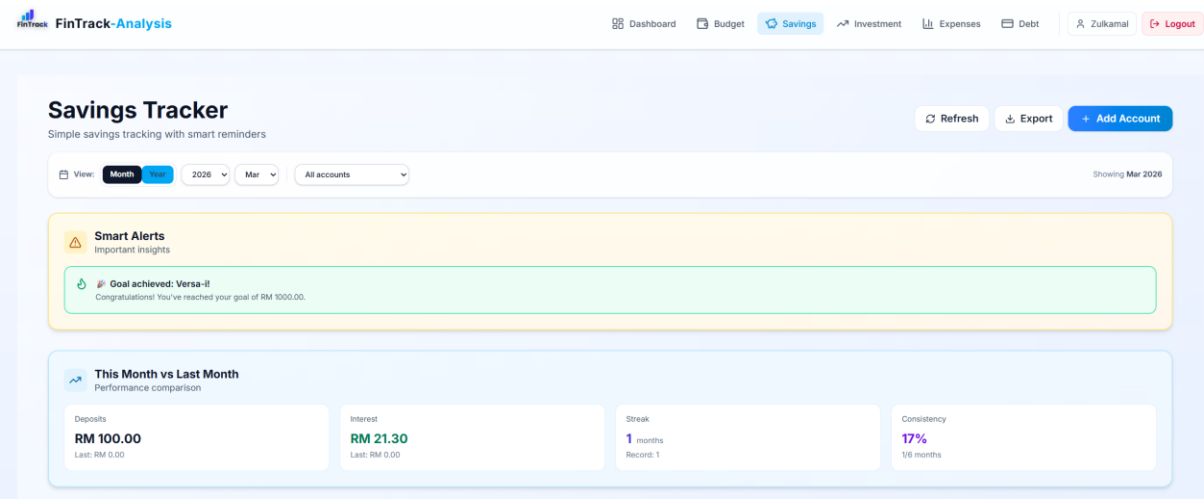
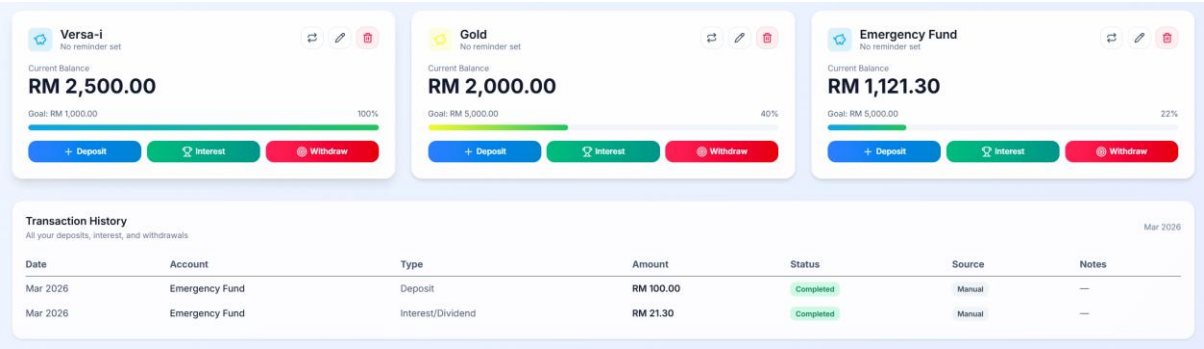
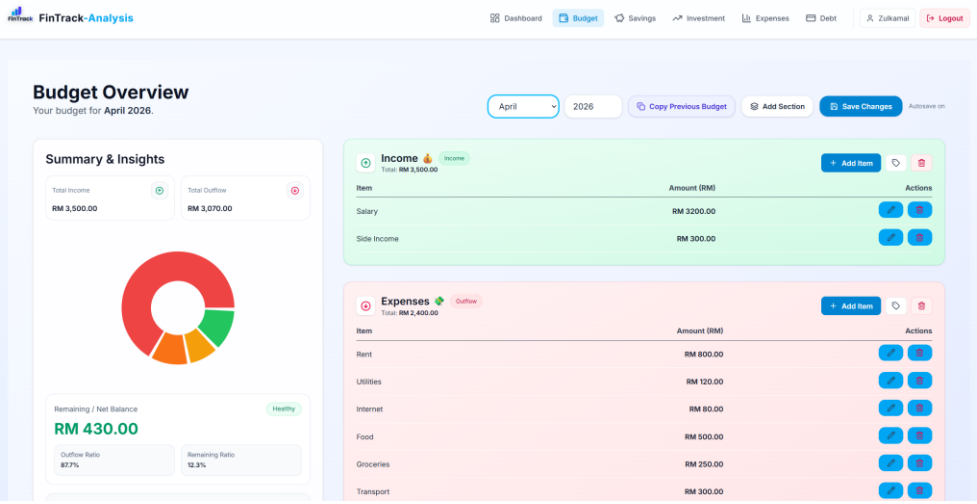


Chart and data card of Saving pages

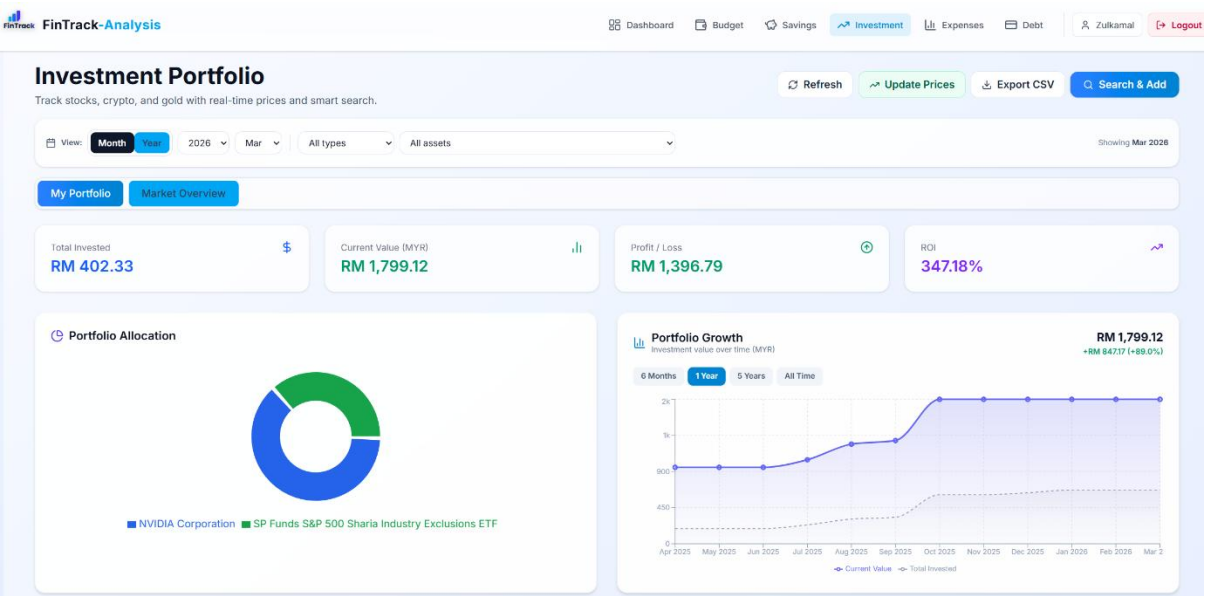
Savings card, goal and savings history



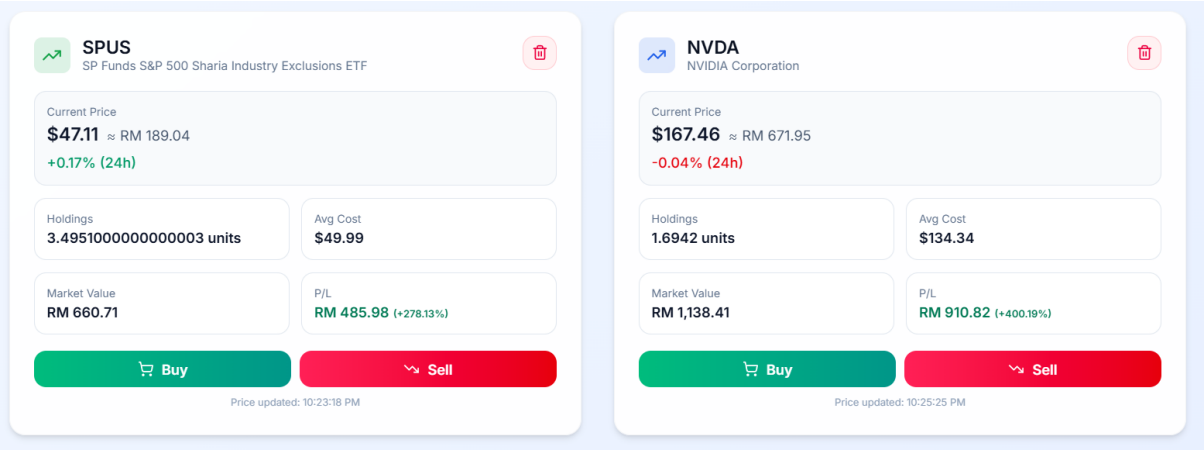
Screen shot of Budgeting Pages



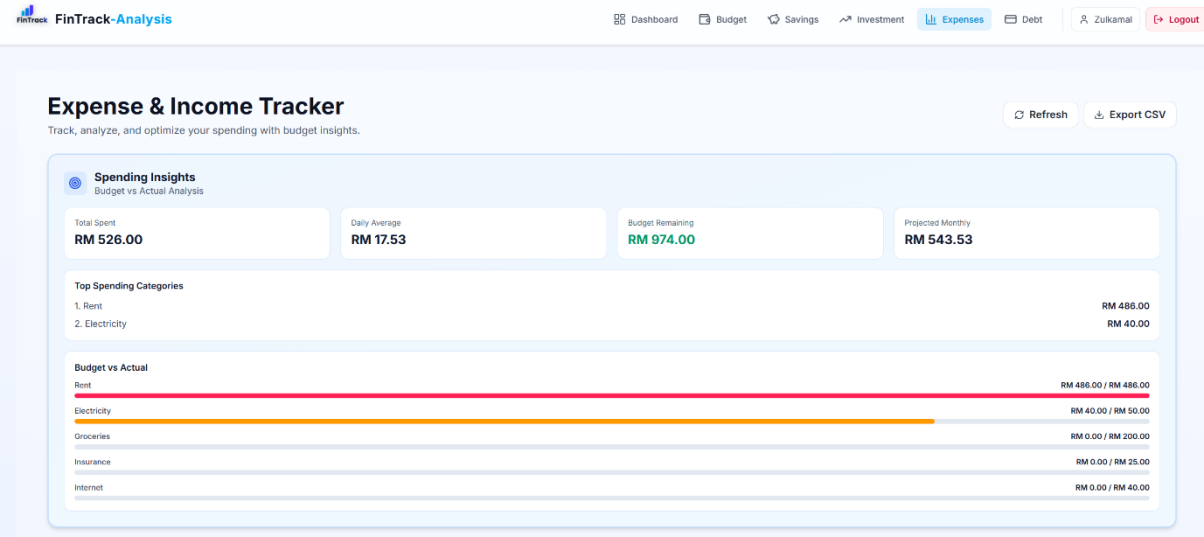
Screen shot of Investment Pages



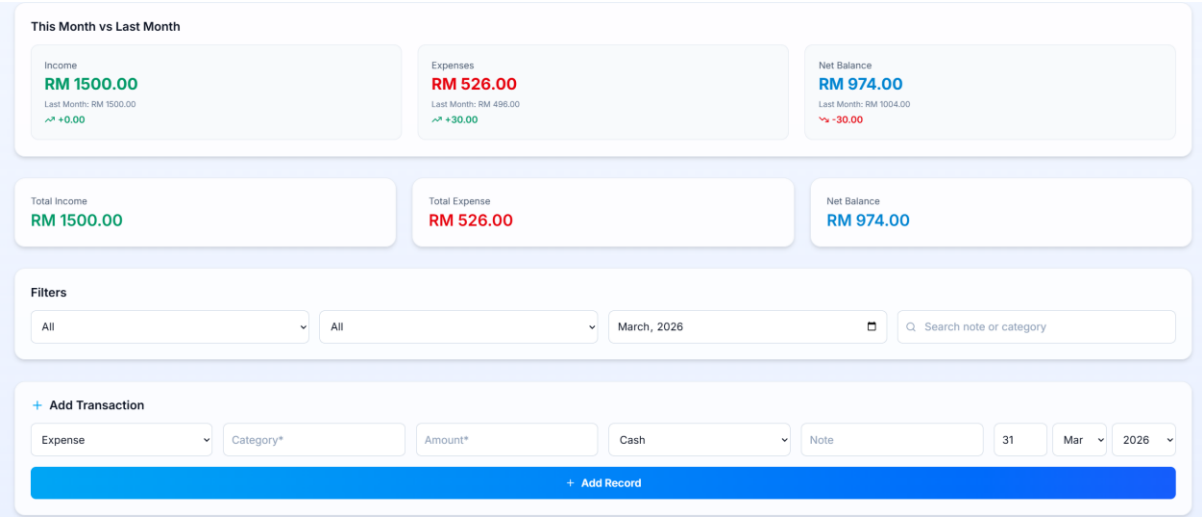
Stock Card of investment pages



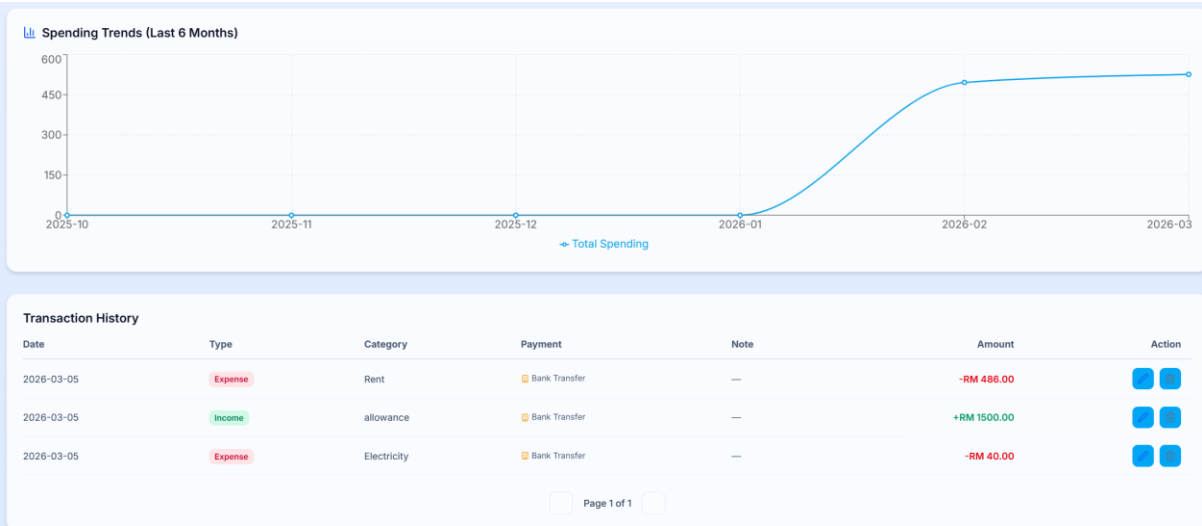
Screen shot of Expense Pages



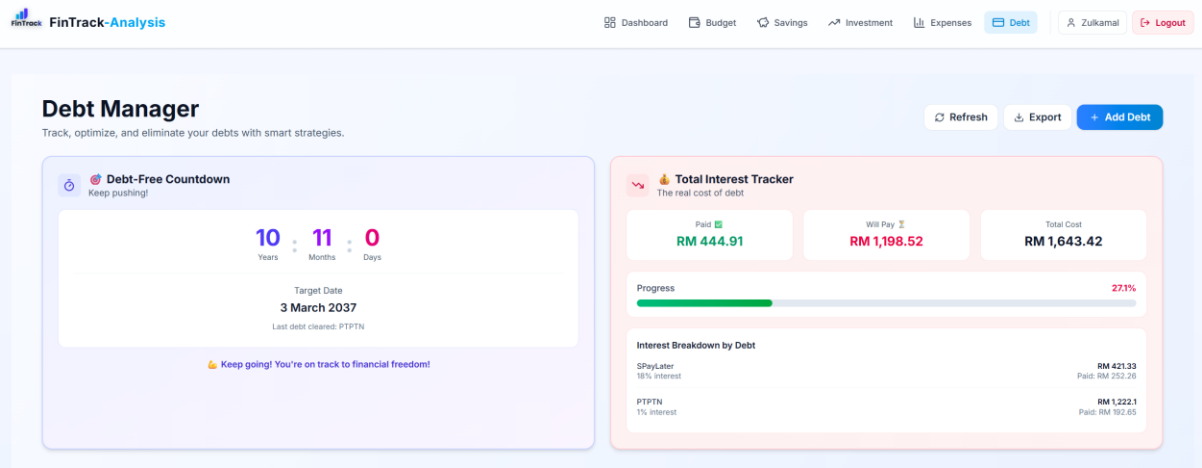
Monthly comparison expense and add new expense



Spending chart and Transaction history



Screen Shot of Debt management pages



Payment calendar and data card of debt pages

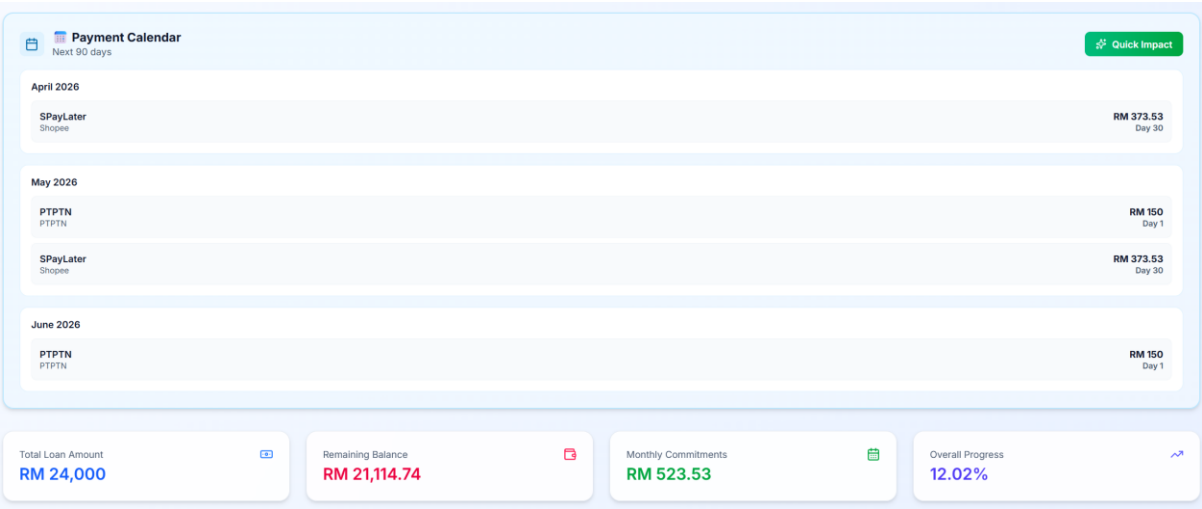
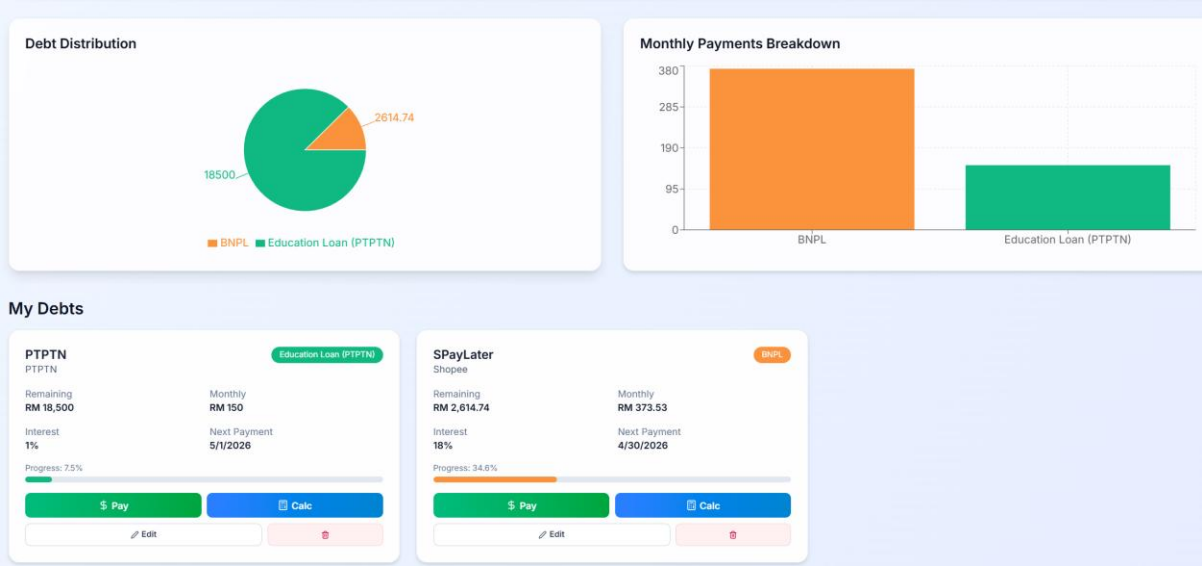


Chart and debt/loan card to make payment and to edit



Appendix B – Coding Snippet

Register user Controller

```
export async function register(req, res) {
  const { name, email, password } = req.body;

  if (!name || !email || !password) {
    return res.status(400).json({ message: "name, email, password required" });
  }

  const normalizedEmail = String(email).trim().toLowerCase();
  const normalizedName = String(name).trim();

  const emailOk = /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(normalizedEmail);
  if (!emailOk)
    return res.status(400).json({ message: "Invalid email format" });

  if (normalizedName.length < 2) {
    return res.status(400).json({ message: "Name too short" });
  }

  if (String(password).length < 8) {
    return res
      .status(400)
      .json({ message: "Password must be at least 8 characters" });
  }

  const existing = await User.findOne({ email: normalizedEmail });
  if (existing)
    return res.status(409).json({ message: "Email already registered" });

  const passwordHash = await bcrypt.hash(password, 12);

  const user = await User.create({
    name: normalizedName,
    email: normalizedEmail,
    passwordHash,
  });
```

Loggin Function

```
export async function login(req, res) {
  const { email, password } = req.body;

  if (!email || !password)
    return res.status(400).json({ message: "email & password required" });

  const user = await User.findOne({ email: email.toLowerCase() });
  if (!user) return res.status(401).json({ message: "Invalid credentials" });

  const ok = await bcrypt.compare(password, user.passwordHash);
  if (!ok) return res.status(401).json({ message: "Invalid credentials" });

  const token = signToken(user._id.toString());
  res.json({
    token,
    user: { id: user._id, name: user.name, email: user.email },
  });
}
```

Dashboard Controller, financial score Calculation

```
// ======================================================================
// 9. FINANCIAL SCORE
// ======================================================================
let financialScore = 50;

if (savingsRate >= 30) financialScore += 15;
else if (savingsRate >= 20) financialScore += 12;
else if (savingsRate >= 10) financialScore += 8;
else if (savingsRate > 0) financialScore += 3;

if (monthsCovered >= 6) financialScore += 15;
else if (monthsCovered >= 3) financialScore += 10;
else if (monthsCovered >= 1) financialScore += 5;

if (totalDebtBalance === 0) financialScore += 10;
else if (debtToIncome < 20) financialScore += 8;
else if (debtToIncome < 35) financialScore += 4;

if (budgetHealth.length > 0) {
  const healthyPercent =
    budgetHealth.filter((b) => b.status === "healthy").length /
    budgetHealth.length;
  financialScore += Math.round(healthyPercent * 10);
}

financialScore = Math.min(financialScore, 100);
```

Savings Account Controller Delete Savings

```
/**
 * Delete a savings account
 */
export async function deleteAccount(req, res) {
  try {
    const userId = req.user.id || req.user._id;
    const { id } = req.params;

    const account = await SavingsAccount.findOne({ _id: id, userId });
    if (!account) {
      return res.status(404).json({ message: "Account not found" });
    }

    // Delete all transactions for this account
    await SavingsTransaction.deleteMany({ accountId: id });

    // Delete the account
    await SavingsAccount.deleteOne({ _id: id });

    logger.info(`Savings account deleted: ${id} by user ${userId}`);
    res.json({ message: "Account deleted successfully" });
  } catch (error) {
    logger.error("Error deleting savings account:", error);
    res.status(500).json({ message: "Failed to delete account" });
  }
}
```

Investment Live Price Service showing multiple fetching from different API

```
/**
 * Get price for any asset type
 */
async getAssetPrice(symbol, type, exchange = "US") {
  try {
    let priceData;

    switch (type) {
      case "crypto":
        priceData = await this.getCryptoPrice(symbol);
        break;
      case "gold":
        priceData = await this.getGoldPrice();
        break;
      case "stock":
      case "etf":
      default:
        priceData = await this.getStockPrice(symbol, exchange);
        break;
    }

    // Get MYR conversion rate
    const myrRate = await this.getMYRRate(priceData.currency);

    return {
      ...priceData,
      myrRate,
    };
  } catch (error) {
    logger.error("Get asset price error:", {
      symbol,
      type,
      exchange,
      error: error.message,
    });
    return {
      price: 0,
      currency: "USD",
      change24h: 0,
      myrRate: 4.5,
      lastUpdated: new Date(),
      error: true,
      errorMessage: error.message,
    };
```

Historical Price auto Fill, show Google Finance inspired features that auto filled by date

```
async getHistoricalPrice(symbol, type, exchange = "US", dateStr) {
  // dateStr format: "YYYY-MM-DD"
  const cacheKey = `historical:${symbol}:${dateStr}`;
  const cached = priceCache.get(cacheKey);
  if (cached) return cached;

  try {
    if (type === "crypto") {
      // CoinGecko historical
      const coinMap = {
        BTC: "bitcoin", ETH: "ethereum", USDT: "tether", BNB: "binancecoin",
        SOL: "solana", XRP: "ripple", ADA: "cardano", DOGE: "dogecoin",
        AVAX: "avalanche-2", MATIC: "matic-network", DOT: "polkadot",
        LINK: "chainlink", UNI: "uniswap", LTC: "litecoin",
      };
      const coinId = coinMap[symbol.toUpperCase()] || symbol.toLowerCase();
      const [y, m, d] = dateStr.split("-");
      const cgDate = `${d}-${m}-${y}`;

      const response = await axios.get(
        `https://api.coingecko.com/api/v3/coins/${coinId}/history`,
        { params: { date: cgDate, localization: false }, timeout: 10000 }
      );

      const price = response.data?.market_data?.current_price?.usd || 0;
      const result = { price, currency: "USD", date: dateStr, source: "coingecko" };
      priceCache.set(cacheKey, result, 86400); // cache 24h
      return result;
    }

    if (type === "gold") {
      // Gold doesn't have great free historical APIs, return 0 to let user input manually
      return { price: 0, currency: "USD", date: dateStr, source: "manual", note: "Historical gold prices unavailable. Please enter manually
    }

    // Stocks/ETFs — Yahoo Finance historical
    const yahooSymbol =
      exchange === "KLSE" && !symbol.endsWith(".KL")
        ? `${symbol}.KL`
        : symbol;

    const queryDate = new Date(dateStr);
```

React Frontend Investment Growth widget (portfolio with dual line chart)

```
// Portfolio Growth Chart (replaces projection)
const PortfolioGrowthWidget = ({ data, range, onRangeChange, loading, currentValue }) => {
  const growthStats = useMemo(() => {
    if (!data || data.length < 2) return { amount: 0, percent: 0 };
    const first = data[0].value;
    const last = data[data.length - 1].value;
    const amount = last - first;
    const percent = first > 0 ? (amount / first) * 100 : 0;
    return { amount, percent };
  }, [data]);

  return (
    <div className="bg-white/95 backdrop-blur-lg rounded-2xl border border-gray-100 p-5 sm:p-6 shadow-md hover:shadow-lg transition">
      <div className="flex flex-col sm:flex-row sm:items-center sm:justify-between gap-3 mb-4">
        <div class="flex items-center gap-2">
          <BarChart3 className="w-5 h-5 text-sky-600" />
          <div>
            <h2 className="text-lg font-semibold text-[#0b1222]">Portfolio Growth</h2>
            <p className="text-xs text-slate-500">Investment value over time (MYR)</p>
          </div>
        </div>
        <div class="text-right">
          <p class="text-lg font-bold text-slate-900">{formatRM(currentValue)}</p>
          {data.length >= 2 && (
            <p class={`text-xs font-semibold ${growthStats.amount >= 0 ? "text-emerald-600" : "text-rose-600"}`}>
              {growthStats.amount >= 0 ? "+" : ""}{formatRM(growthStats.amount)} ({growthStats.percent >= 0 ? "+" : ""}{growthStats.percent.toFixed(1)}%)
            </p>
          )}
        </div>
      </div>

      {/* Range Tabs */}
      <div class="flex flex-wrap gap-1.5 mb-4">
        {RANGE_TABS.map((tab) => (
          <button key={tab.value} type="button" onClick={() => onRangeChange(tab.value)}
            class={`px-3 py-1.5 rounded-lg text-xs font-semibold transition ${range === tab.value ? "bg-sky-600 text-white shadow-sm" : "bg-slate-100 text-slate-600 hover:bg-slate-200
            {tab.label}
          </button>
        ))}
      </div>

      <div class="h-[280px]">
        {loading ? (
```

Mongoose Schema(Budget model with Section and Item)

backend > src > models > Budget.js > BudgetSectionSchema

```
9   );
5     label: { type: String, required: true, trim: true, maxlength: 80 },
6     amount: { type: Number, required: true, min: 0 },
7   },
8   { _id: false }
9 );
10
11 const BudgetSectionSchema = new mongoose.Schema(
12   {
13     name: { type: String, required: true, trim: true, maxlength: 40 },
14
15
16     kind: {
17       type: String,
18       enum: ["income", "expense"],
19       required: true,
20       default: "expense",
21       index: true,
22     },
23
24
25     type: {
26       type: String,
27       enum: ["income", "expense", "saving", "debt"],
28       default: "expense",
29       index: true,
30     },
31
32     items: { type: [BudgetItemSchema], default: [] },
33   },
34   { _id: false }
35 );
36
37 const BudgetSchema = new mongoose.Schema(
38   {
39     userId: {
40       type: mongoose.Schema.Types.ObjectId,
41       ref: "User",
42       index: true,
43       required: true,
44     },
45     year: { type: Number, index: true, required: true },
46     month: { type: Number, index: true, required: true, min: 1, max: 12 }, // 1-12
47     sections: { type: [BudgetSectionSchema], default: [] },
48   },
49   { timestamps: true }
```